

SLEZSKÁ UNIVERZITA V OPAVĚ

Obchodně podnikatelská fakulta v Karviné

Možnosti paralelizace výpočtů pomocí grafických karet

Diplomová práce

SLEZSKÁ UNIVERZITA V OPAVĚ

Obchodně podnikatelská fakulta v Karviné

Studijní program: Manažerská informatika

Bc. Jakub Kubisz

Možnosti paralelizace výpočtů pomocí grafických karet

Parallelization of Computations Using Graphics Cards

Diplomová práce

Karviná 2026

Vedoucí diplomové práce:
doc. Ing. Jan Górecki, Ph.D.

Abstrakt

Diplomová práce se zabývá možnostmi paralelizace výpočtů. V teoretické části jsou popsány základní principy paralelních výpočtů, rozdíly mezi sekvenčním a paralelním přístupem, typy paralelismu a také Amdahlův i Gustafsonův zákon. Dále je popsána, architektura GPU a přehled dostupných technologií pro GPU výpočty včetně CUDA, OpenCL a nástrojů pro C#/.NET. V praktické části jsou implementovány dvě výpočetně náročné úlohy, násobení matic a celulární automat Game of Life a to ve třech variantách: sekvenční CPU, paralelní CPU a GPU s využitím knihoven ILGPU a ComputeSharp v jazyce C#. Práce prezentuje experimenty slabé a silné škálovatelnosti pro CPU variantu, které jsou porovnány s predikcemi Amdahlova a Gustafsonova zákona. Pro GPU variantu práce měří závislosti výkonu na velikosti vstupních dat, porovnání naměřených dat z obou C# knihoven a identifikaci crossover pointů mezi CPU a GPU.

Abstract

This thesis explores the possibilities of parallelizing computations. The theoretical section describes the basic principles of parallel computing, the differences between sequential and parallel approaches, types of parallelism, as well as Amdahl's and Gustafson's laws. It also describes GPU architecture and provides an overview of available technologies for GPU computing, including CUDA, OpenCL, and tools for C#/.NET. The practical section implements two computationally intensive tasks, matrix multiplication and the Game of Life cellular automaton, in three variants: sequential CPU, parallel CPU, and GPU using the ILGPU and ComputeSharp libraries in C#. The thesis presents experiments on weak and strong scalability for the CPU variant, which are compared with predictions based on Amdahl's and Gustafson's laws. For the GPU variant, the thesis measures performance dependencies on input data size, compares measured data from both C# libraries, and identifies crossover points between CPU and GPU.

Obsah

Úvod	1
1 Paralelní výpočty	2
1.1 Sekvenční vs paralelní přístup	2
1.2 Typy paralelismu	2
1.3 Flynnova taxonomie	3
1.4 Amdahlův zákon	4
1.5 Gustafsonův zákon	5
1.6 Metriky paralelního výkonu	6
2 Architektura GPU	8
2.1 Historický vývoj GPU	8
2.2 Architektonické rozdíly CPU a GPU	9
2.3 Paměťová hierarchie GPU	10
2.4 Execution model GPU	12
2.5 Přenos dat mezi CPU a GPU	13
3 Technologie a nástroje pro GPU výpočty	14
3.1 CUDA	14
3.2 OpenCL	15
3.3 Vulkan Compute	16
3.4 Nástroje v prostředí C# / .NET	17
3.5 Nástroje v prostředí Python	20
3.6 Srovnání technologií	21
4 Návrh experimentu	22
4.1 Výběr úloh	22
4.2 Testovací prostředí	23
4.3 Metodika měření	24
4.4 Očekávané výstupy	27
5 Implementace	28
5.1 Architektura řešení	28

5.2	Násobení matic	29
5.3	Game of Life	33
5.4	Měřicí framework	35
5.5	GPU implementace v ComputeSharp	35
6	Výsledky měření	38
6.1	Násobení matic	38
6.2	Game of Life	44
6.3	Souhrnné srovnání CPU a GPU	49
6.4	Porovnání s Amdahlovým zákonem	50
6.5	Porovnání s Gustafsonovým zákonem	53
6.6	Porovnání ILGPU a ComputeSharp	58
7	Diskuse	60
7.1	Hlavní zjištění	60
7.2	CPU versus GPU – crossover point	60
7.3	Vliv overheadu přenosu dat CPU–GPU	61
7.4	Anomálie v datech	61
7.5	Interpretace Amdahlova zákona	63
7.6	Interpretace Gustafsonova zákona	63
7.7	Srovnání ILGPU a ComputeSharp	64
7.8	Srovnání C# ekosystému s alternativami	65
7.9	Omezení práce	65
7.10	Možná rozšíření a budoucí práce	66
	Závěr	67
	Seznam použité literatury	69
	Seznam tabulek	71
	Seznam obrázků	73
	Seznam zkratk	74
	Seznam příloh	75
	Přílohy	76

A	Zdrojové kódy	76
A.1	Rozhraní IBenchmark	76
A.2	Násobení matic – sekvenční verze	76
A.3	Násobení matic – paralelní CPU verze	78
A.4	Násobení matic – GPU verze	80
A.5	Game of Life – sekvenční verze	83
A.6	Game of Life – paralelní CPU verze	85
A.7	Game of Life – GPU verze	88
A.8	Třída BenchmarkRunner	91
A.9	Násobení matic – ComputeSharp verze	94
A.10	Game of Life – ComputeSharp verze	96
B	Podrobné výsledky měření	99
B.1	Násobení matic – kompletní časy	99
B.2	Násobení matic – speedup a efektivita	101
B.3	Game of Life – kompletní časy	103
B.4	Game of Life – speedup a efektivita	104
B.5	Amdahlova analýza – strong-scaling experiment	105
B.6	Gustafsonova analýza – weak-scaling experiment	105
B.7	Konfigurace testovacího prostředí	107
C	Datové soubory experimentů	108
D	Benchmarková aplikace	109

Úvod

V posledních desetiletích se výpočetní nároky dramaticky zvýšily, zejména v oblastech umělé inteligence nebo zpracování obrazu. Tyto úlohy mají společné to, že vyžadují zpracování obrovského množství dat v co nejkratším čase. Tradiční sekvenční přístup, kdy procesor vykonává jednu instrukci za druhou, přitom naráží na fyzikální limity. Zvyšování taktovací frekvence se prakticky zastavilo kolem roku 2005 kvůli problémům s odváděním tepla a spotřebou energie (Hennessy & Patterson, 2017).

Jednou z cest, jak tento limit obejít, je rozložit výpočet mezi více jader. Moderní procesory tak místo jednoho jádra obsahují několik jader, které pracují současně. Grafické karty (GPU) tento princip dovedly ještě dál, původně vznikly pro vykreslování 3D grafiky, ale dnes obsahují tisíce jednoduchých výpočetních jader. Díky tomu jsou GPU ideální pro úlohy, kde je potřeba provést stejnou operaci na velkém množství dat najednou, takzvaný datový paralelismus (Kirk & Hwu, 2016).

Cílem diplomové práce je prozkoumat možnosti paralelizace výpočtů, které nabízejí moderní programovací jazyky a technologie. V praktické části jsou vybrané výpočetně náročné úlohy implementovány ve třech variantách (sekvenční na CPU, paralelní na CPU a paralelní na GPU). U CPU paralelní varianty je měřena závislost zrychlení na počtu zapojených vláken (1 až 12) a analyzována pomocí Amdahlova a Gustafsonova zákona. U GPU implementace jsou porovnány dvě knihovny (ILGPU a ComputeSharp), je měřena závislost výkonu na velikosti vstupních dat, zrychlení oproti CPU včetně identifikace crossover pointů. Pro implementaci je zvolen jazyk C# s knihovnami ILGPU a ComputeSharp, které umožňují psát GPU výpočty přímo v prostředí .NET - ILGPU přes CUDA backend, ComputeSharp přes DirectX 12. Výsledky práce mohou sloužit jako praktický průvodce pro vývojáře v .NET prostředí, kteří zvažují využití GPU výpočtů ve svých projektech.

Teoretická část práce nejprve zavádí základní pojmy paralelních výpočtů včetně Amdahlova a Gustafsonova zákona, které definují teoretické limity zrychlení. Následně se věnuje architektuře grafických karet, ukazuje, v čem se liší od klasických procesorů, a podává přehled dostupných technologií pro GPU výpočty od CUDA a OpenCL přes nástroje v C# až po Python. Praktická část popisuje návrh experimentu, výběr úloh a metodiku měření, pokračuje detailním rozбором implementace a představuje naměřené výsledky spolu s jejich interpretací. Závěr shrnuje hlavní zjištění práce.

1 Paralelní výpočty

Paralelní výpočty představují hlavní přístup k řešení výpočetně náročných úloh v situaci, kdy nárůst výkonu jednotlivých procesorových jader narazil na fyzikální limity. Aby bylo možné pochopit principy GPU paralelizace, je nejprve nutné projít základní pojmy, od rozdílu mezi sekvenčním a paralelním zpracováním přes Flynnovu klasifikaci architektur až po Amdahlův a Gustafsonův zákon, které definují teoretické limity dosažitelného zrychlení.

1.1 Sekvenční vs paralelní přístup

Klasický sekvenční výpočet funguje tak, že procesor vykonává jednu instrukci za druhou v přesně daném pořadí. Program má jeden tok řízení a v každém okamžiku se děje právě jedna věc. Tento model odpovídá von Neumannově architektuře, která dominuje výpočetní technice již od jejích počátků (Hennessy & Patterson, 2017).

Problém nastává ve chvíli, kdy samotné zvyšování rychlosti jednoho procesoru naráží na fyzikální limity. Jak popsal Gordon Moore (1965) ve svém známém pozorování, počet tranzistorů na čipu se přibližně každé dva roky zdvojnásobuje. Dlouhou dobu to znamenalo i odpovídající nárůst taktovací frekvence a výkonu.

Ovšem kolem roku 2005, se tento trend výrazně zpomalil, protože procesory narazily na takzvanou „power wall“, tedy hranici, za kterou další zvyšování frekvence vede k nepřijatelnému nárůstu spotřeby energie a generování tepla. Výrobci proto začali přecházet k vícejádrovým architekturám jako k efektivnější cestě dalšího zvyšování výkonu (Hennessy & Patterson, 2017). Tím vznikla potřeba paralelního programování, tedy takového přístupu, kdy se úloha rozdělí na menší části a ty se zpracovávají současně na více procesorech nebo jádrech.

Paralelní výpočet ale není jen o tom rozdělit práci mezi více jader. Vždy existuje část programu, která se musí provést sekvenčně, například inicializace, synchronizace nebo závislosti mezi kroky výpočtu. Právě poměr mezi paralelizovatelnou a sekvenční částí programu určuje, jakého reálného zrychlení lze dosáhnout (Trobec et al., 2018).

1.2 Typy paralelismu

V oblasti paralelního programování se nejčastěji rozlišují dva základní typy paralelismu, které se liší způsobem, jakým je práce rozložena mezi jednotlivé výpočetní jednotky.

1.2.1 Datový paralelismus

Datový paralelismus (*data parallelism*) spočívá v tom, že se stejná operace provádí současně na různých částech dat. Typickým příkladem je situace, kdy máme velké pole čísel a potřebujeme ke každému prvku přičíst konstantu. Místo toho, abychom procházeli pole prvek po prvku, můžeme práci rozdělit mezi více výpočetních jednotek a každá zpracuje svou část pole (Kirk & Hwu, 2016).

Datový paralelismus je přirozeným modelem pro GPU výpočty, protože grafické karty jsou navrženy právě pro tento typ práce. Při vykreslování obrazu se stejný shader (program pro zpracování pixelu) spouští pro miliony pixelů současně a to je v podstatě datový paralelismus (Owens et al., 2008).

1.2.2 Úlohový paralelismus

Úlohový paralelismus (*task parallelism*) oproti tomu znamená, že různá vlákna nebo procesory provádějí různé činnosti, případně na různých datech. Například v jednom vlákne se načítají data ze souboru, ve druhém se provádí výpočet a ve třetím se zapisují výsledky. Každé vlákno dělá jinou věc, ale běží současně (Herlihy et al., 2020).

V praxi se oba přístupy často kombinují. Například při trénování neuronové sítě se datový paralelismus používá pro zpracování dávky trénovacích vzorků na GPU, zatímco úlohový paralelismus může zajišťovat paralelní načítání dat z disku do paměti.

1.3 Flynnova taxonomie

V roce 1972 navrhl Michael Flynn (1972) klasifikaci počítačových architektur na základě dvou kritérií, počtu toků instrukcí a počtu toků dat. Tato klasifikace, známá jako Flynnova taxonomie, rozlišuje čtyři základní kategorie.

1.3.1 SISD – Single Instruction, Single Data

Architektura SISD odpovídá klasickému sekvenčnímu procesoru. V každém taktu se provádí jedna instrukce na jednom datovém prvku. Jedná se o nejjednodušší model, který odpovídá tradiční von Neumannově architektuře. Většina běžných programů je napsána pro tento model (Flynn, 1972).

1.3.2 SIMD – Single Instruction, Multiple Data

V architektuře SIMD se jedna instrukce provádí současně na více datových prvcích. Příkladem jsou vektorové instrukce moderních procesorů (SSE, AVX), které dokáží jednou instrukcí například sečíst čtyři nebo osm čísel najednou (Hennessy & Patterson, 2017).

SIMD je velmi efektivní pro pravidelné operace nad poli dat, jako je zpracování obrazu, zvuku nebo vědecké výpočty. Omezením je, že všechny zpracovávané prvky musí procházet stejnou instrukcí, a pokud by některé vyžadovaly jinou operaci, dochází ke ztrátě výkonu.

1.3.3 MIMD – Multiple Instruction, Multiple Data

MIMD je nejobecnější model, kde více procesorů¹ provádí různé instrukce na různých datech. Spadají sem moderní víceprocesorové systémy, víceprocesorové servery i počítačové clustery. Každý procesor může mít svůj vlastní program a pracovat na jiné části úlohy (Flynn, 1972).

Moderní vícejádrové procesory (CPU) pracují v režimu MIMD, kde každé jádro může vykonávat jiný program nebo jiné vlákno stejného programu.

1.3.4 SIMT – Single Instruction, Multiple Threads

Flynnova taxonomie ve svém původním znění neobsahuje kategorii SIMT, protože v roce 1972 GPU v dnešní podobě neexistovaly. SIMT je koncept zavedený firmou NVIDIA pro popis způsobu, jakým pracují jejich grafické karty (Lindholm et al., 2008).

V modelu SIMT se stejná instrukce spouští na mnoha vláknech, přičemž každé vlákno pracuje s jinými daty. Na rozdíl od čistého SIMD ale vlákna mohou částečně divergovat (*branch divergence*), některá vlákna mohou provést jednu větev podmínky, jiná druhou. GPU to řeší tak, že vlákna v rámci jedné skupiny (warpu) vykonají obě větve postupně, vlákna která do právě vykonávané větve nepatří, jsou dočasně maskována, tedy nevykonávají instrukce a čekají na dokončení aktivní větve. To sice snižuje efektivitu, ale umožňuje flexibilnější programování než čisté SIMD (NVIDIA Corporation, 2024a).

1.4 Amdahlův zákon

Jedním z nejdůležitějších teoretických principů v oblasti paralelních výpočtů je Amdahlův zákon, formulovaný Gene Amdahl (1967). Tento zákon stanovuje horní hranici zrychlení, kterého lze dosáhnout paralelizací programu.

Základní myšlenka je jednoduchá, každý program se skládá ze dvou částí, z nichž jednu lze paralelizovat a druhá musí běžet sekvenčně (například inicializace, čtení vstupních dat, synchronizace mezi vlákny). I při libovolném zrychlení paralelizovatelné části sekvenční část zůstává stejná a omezuje celkové zrychlení.

Matematicky se Amdahlův zákon vyjadřuje následovně. Označíme-li P podíl doby výpočtu, který lze paralelizovat, a N počet procesorů, pak maximální dosažitelné zrychlení S je:

$$S = \frac{1}{(1 - P) + \frac{P}{N}} \quad (1.1)$$

kde $(1 - P)$ je podíl sekvenční části programu (Amdahl, 1967).

¹V kontextu Flynnovy taxonomie je pojem procesor chápán jako samostatná výpočetní jednotka schopná nezávisle vykonávat vlastní tok instrukcí; nemusí tedy označovat pouze fyzický CPU čip.

Aby bylo vidět, jaký vliv má sekvenční část na celkové zrychlení, ukažme si několik příkladů. Pokud lze paralelizovat 90 % výpočtu ($P = 0,9$) a máme k dispozici 10 procesorů, dosáhneme zrychlení:

$$S = \frac{1}{0,1 + \frac{0,9}{10}} = \frac{1}{0,19} \approx 5,26$$

I přesto, že máme 10 procesorů, zrychlení je pouze přibližně pětinasobné. A i kdybychom měli nekonečný počet procesorů ($N \rightarrow \infty$), zrychlení by nikdy nepřesáhlo:

$$S_{\max} = \frac{1}{1 - P} = \frac{1}{0,1} = 10$$

To znamená, že 10 % sekvenčního kódu omezuje maximální zrychlení na 10×, bez ohledu na to, kolik procesorů přidáme (Hill & Marty, 2008).

Pro praxi z toho plyne, že nemá smysl přidávat další a další procesory, pokud existuje nezanedbatelná sekvenční část programu. Účinnější je snažit se zmenšit sekvenční podíl výpočtu.

1.5 Gustafsonův zákon

Amdahlův zákon bývá někdy kritizován za to, že je příliš pesimistický. John Gustafson (1988) navrhl alternativní pohled na problém. Hlavní myšlenka je, že v praxi se s přibývajícím počtem procesorů obvykle zvětšuje i velikost řešeného problému, ne se pouze řeší stejný problém rychleji.

Gustafsonův zákon zavádí pojem *škálovaného zrychlení* (scaled speedup). Předpokládejme, že paralelní program běží na N procesorech a sekvenční část (ta, která nemůže být paralelizována) tvoří podíl α celkového času paralelního běhu. Pak škálované zrychlení je:

$$S(N) = N - \alpha \cdot (N - 1) \tag{1.2}$$

Na rozdíl od Amdahla, Gustafson ukazuje, že zrychlení roste téměř lineárně s počtem procesorů, pokud odpovídajícím způsobem zvětšujeme velikost problému. To odpovídá reálné praxi, protože když dostaneme k dispozici výkonnější hardware, typicky chceme řešit větší úlohy. Například místo simulace s tisícem částic spustíme simulaci s milionem částic, místo trénování malé neuronové sítě trénujeme velkou (Gustafson, 1988).

Rozdíl mezi oběma zákony je tedy v tom, co považují za konstantní:

- **Amdahl** – konstantní velikost úlohy, jak moc ji zrychlíme přidáním procesorů (silné škálování),
- **Gustafson** – konstantní čas výpočtu, jak velkou úlohu zvládneme s více procesory (slabé škálování).

Tyto přístupy se vzájemně nevylučují a pro kompletní analýzu je vhodné pracovat s oběma. V praktické části této práce budou naměřená data z CPU paralelizace porovnána s oběma modely pro analýzu škálování na vícejádrovém procesoru.

1.6 Metriky paralelního výkonu

Pro hodnocení účinnosti paralelizace se v praxi používá několik základních metrik.

1.6.1 Zrychlení (Speedup)

Zrychlení je základní metrikou a vyjadřuje, kolikrát je paralelní řešení rychlejší než sekvenční. Definuje se jako poměr doby sekvenčního výpočtu T_1 ku době paralelního výpočtu T_p na p procesorech (Trobac et al., 2018):

$$S(p) = \frac{T_1}{T_p} \quad (1.3)$$

Ideální (lineární) zrychlení nastává, když $S(p) = p$, tedy když zdvojnásobení počtu procesorů zdvojnásobí rychlost. V praxi bývá zrychlení téměř vždy nižší než lineární kvůli sekvenčním částem programu, komunikačnímu overheadu a synchronizaci mezi vlákny.

Ve výjimečných případech může nastat takzvané superlineární zrychlení ($S(p) > p$), což je obvykle způsobeno lepším využitím cache paměti, protože při rozdělení dat mezi více procesorů se do cache každého procesoru vejde větší podíl pracovních dat (Trobac et al., 2018).

1.6.2 Efektivita

Efektivita vyjadřuje, jak dobře jsou jednotlivé procesory využity. Jedná se o zrychlení vydělené počtem procesorů (Trobac et al., 2018):

$$E(p) = \frac{S(p)}{p} = \frac{T_1}{p \cdot T_p} \quad (1.4)$$

Ideální efektivita je $E = 1$ (100 %), což by znamenalo, že každý procesor přispívá svým plným dílem k celkovému zrychlení. V praxi efektivita klesá s rostoucím počtem procesorů, protože roste podíl komunikačního a synchronizačního overheadu.

1.6.3 Škálovatelnost

Škálovatelnost popisuje, jak se chová výkon systému při změně jeho velikosti. Rozlišují se dva základní typy:

Silná škálovatelnost – velikost úlohy je pevná a přidáváme procesory. Cílem je zmenšit dobu výpočtu. Odpovídá pohledu Amdahlova zákona.

Slabá škálovatelnost – s přidáním procesorů úměrně zvětšujeme i velikost úlohy. Cílem je, aby doba výpočtu zůstala konstantní. Odpovídá pohledu Gustafsonova zákona.

Dobrá škálovatelnost je podstatnou vlastností paralelního programu, protože určuje, jak účinně dokáže využít rostoucí výpočetní zdroje. U GPU výpočtů bývá škálovatelnost výborná hlavně u datově paralelních úloh, kde každý prvek dat vyžaduje nezávislý výpočet.

2 Architektura GPU

Grafické karty se z čistě grafického hardwaru postupně vyvinuly v univerzální paralelní procesory, které dnes pohánějí vše od strojového učení po vědecké simulace. Pro pochopení, proč jsou GPU pro určité typy úloh výrazně rychlejší než CPU, je nutné podívat se na jejich vnitřní architekturu, od historického vývoje přes paměťový model až po způsob organizace vláken.

2.1 Historický vývoj GPU

2.1.1 Od grafického renderingu k GPU

Grafické procesory vznikly v 80. a 90. letech 20. století jako specializovaný hardware pro vykreslování 2D a později 3D grafiky. Jejich hlavním úkolem bylo odlehčit hlavnímu procesoru od náročných grafických operací, jako je rasterizace, texturování a transformace vrcholů (Owens et al., 2008).

Důležitým milníkem byl přechod od pevné grafické pipeline (fixed-function pipeline) k programovatelným shaderům na počátku tohoto tisíciletí. Programovatelné shadery umožnily vývojářům psát vlastní programy, které GPU spouštělo pro každý pixel nebo vrchol na obrazovce. Tím se z GPU stal de facto programovatelný paralelní procesor, i když stále zaměřený primárně na grafiku (Buck et al., 2004).

Výzkumníci si brzy všimli, že programovatelnost shaderů lze zneužít i pro negrafické výpočty. Tento přístup, označovaný jako GPGPU (General-Purpose Computing on Graphics Processing Units), ale vyžadoval formulovat výpočetní problémy v termínech textur a fragmentů, což bylo velmi nepraktické (Owens et al., 2008).

2.1.2 Milníky vývoje

Skutečný průlom přišel v roce 2006, kdy NVIDIA představila architekturu Tesla (G80), první GPU s unifikovanými shadery a architekturou přímo navrženou pro obecné výpočty. O rok později vydala NVIDIA platformu CUDA, která umožnila programovat GPU v jazyce podobném C/C++ bez nutnosti používat grafické API (Nickolls et al., 2008).

Následoval rychlý vývoj dalších generací. Fermi (2010) přinesla podporu ECC paměti a lepší podporu double precision, Kepler (2012) zlepšila energetickou účinnost, Maxwell (2014) a Pascal (2016) pokračovaly v trendu zvyšování výkonu. Architektury Volta (2017) a Turing (2018) přidaly specializované tensor cores pro výpočty strojového učení. Současné architektury Ampere (2020), Ada Lovelace (2022) a Blackwell (2024) pokračují v tomto vývoji (NVIDIA Corporation, 2020).

AMD jako hlavní konkurent NVIDIA vytvořila svou vlastní platformu ROCm a architektury RDNA/CDNA. Intel vstoupil na trh s diskretními GPU řady Arc a platformou oneAPI. Nicméně NVIDIA si díky CUDA a rozsáhlému ekosystému udržuje dominantní pozici zejména v oblasti vědeckých a AI výpočtů (Kirk & Hwu, 2016).

2.2 Architektonické rozdíly CPU a GPU

Hlavní rozdíl mezi CPU a GPU spočívá v tom, k čemu byly navrženy. CPU je optimalizované pro nízkou latenci, snaží se co nejrychleji dokončit jednu úlohu. GPU je naopak optimalizované pro vysokou propustnost a zpracovává co nejvíce úloh za jednotku času (Kirk & Hwu, 2016).

2.2.1 Návrh CPU – malý počet výkonných jader

Moderní CPU má typicky 4 až 24 jader (u serverových procesorů i více). Každé jádro je velmi složité a obsahuje rozsáhlé mechanismy pro zrychlení sekvenčního zpracování: hlubokou pipeline, predikci skoků, out-of-order execution, velké cache paměti. Díky tomu je jedno jádro CPU velmi rychlé při zpracování složitěho kódu s mnoha podmínkami a nepravidelnými přístupy do paměti (Hennessy & Patterson, 2017).

Velkou část plochy čipu zabírají cache paměti (L1, L2, L3) a řídicí logika. Samotné výpočetní jednotky (ALU) tvoří relativně malou část čipu.

2.2.2 Návrh GPU – tisíce jednoduchých jader

GPU přistupuje k problému úplně jinak. Místo několika složitých jader obsahuje tisíce jednoduchých jader seskupených do takzvaných streaming multiprocessorů (SM). Jednotlivá jádra GPU jsou mnohem jednodušší než jádra CPU, nemají predikci skoků, mají menší cache a omezenou schopnost out-of-order execution (Kirk & Hwu, 2016).

Co jim ale chybí na sofistikovanosti, dohánějí počtem. Současná GPU nejvyšší třídy obsahují více než 10 000 CUDA jader (u NVIDIA) nebo stream procesorů (u AMD). Většina plochy čipu je věnována výpočetním jednotkám, nikoliv cache pamětem (Kirk & Hwu, 2016).

2.2.3 Srovnání architektur

Rozdíl mezi CPU a GPU lze shrnout následovně:

Tabulka č. 2.1: Srovnání architektur CPU a GPU

Vlastnost	CPU	GPU
Počet jader	jednotky–desítky	tisíce
Složitost jádra	vysoká	nízká
Optimalizace	nízká latence	vysoká propustnost
Cache paměti	velké (MB)	malé (KB na SM)
Vhodné úlohy	sekvenční, větvící se kód	datově paralelní
Paměťová propustnost	desítky GB/s	stovky GB/s–TB/s

Zdroj: vlastní zpracování

GPU jsou tedy ideální pro úlohy, kde se dá stejná operace aplikovat na velké množství dat (datový paralelismus), zatímco CPU jsou lepší pro úlohy s komplexní řídicí logikou a nepravidelnými přístupy do paměti.

2.3 Paměťová hierarchie GPU

Práce s pamětí patří mezi nejdůležitější aspekty GPU programování, protože právě přístupy do paměti bývají hlavní bottleneck výkonu. GPU nabízí několik typů paměti s různými vlastnostmi (NVIDIA Corporation, 2024a).¹

¹Použité pojmy vlákno, warp, blok a SM jsou podrobněji vysvětleny v sekce 2.4.

2.3.1 Globální paměť

Globální paměť je největší paměť na GPU (typicky 4–48 GB u moderních karet). Je přístupná všem vláknům na celém GPU, ale má relativně vysokou latenci (stovky taktů). Pro dosažení maximální propustnosti je nutné, aby vlákna v rámci jednoho warpu (skupiny 32 vláken) přistupovala k sousedním adresám v paměti, takzvaný *coalesced access*. Při nesouvislých přístupech výkon dramaticky klesá (Kirk & Hwu, 2016).

2.3.2 Sdílená paměť

Sdílená paměť (shared memory) je malá, ale velmi rychlá paměť sdílená všemi vlákny v rámci jednoho bloku. Její velikost je typicky 48–228 KB na jeden streaming multiprocessor (SM, výpočetní blok GPU) v závislosti na architektuře (NVIDIA Corporation, 2024a). Z hlediska latence se blíží registrům a je řádově rychlejší než globální paměť.

Sdílená paměť se typicky používá jako uživatelem řízená cache, kdy programátor do ní explicitně kopíruje data z globální paměti a následně s nimi pracuje. To je klíčová optimalizační technika v GPU programování, například při násobení matic (technika zvaná *tiling*).

2.3.3 Registrová paměť

Každé vlákno má k dispozici svou privátní registrovou paměť. Registry jsou nejrychlejší paměti na GPU, ale jejich počet je omezený (typicky 255 32-bitových registrů na vlákno). Pokud výpočet vyžaduje více registrů, než je k dispozici, dochází k takzvanému *register spilling*, kdy se přebytečné hodnoty ukládají do pomalejší lokální paměti (NVIDIA Corporation, 2024a).

2.3.4 Konstantní a texturová paměť

Konstantní paměť je speciální typ paměti optimalizovaný pro případy, kdy všechna vlákna čtou stejnou hodnotu. Má vlastní cache a při správném použití nabízí velmi nízkou latenci. Její velikost je omezena na 64 KB (Kirk & Hwu, 2016).

Texturová paměť je optimalizovaná pro 2D prostorovou lokalitu, tedy pro případy, kdy vlákna přistupují k datům, která jsou blízko sebe ve 2D prostoru. Původně byla určena pro čtení textur při renderování, ale lze ji využít i pro obecné výpočty.

2.3.5 Unifikovaná paměť

CUDA podporuje takzvanou unifikovanou paměť (unified memory), která vytváří jednotný adresní prostor pro CPU i GPU. Programátor nemusí explicitně kopírovat data mezi CPU a GPU paměti, systém to řeší automaticky. To zjednodušuje programování, ale může mít negativní dopad na výkon kvůli implicitním přenosům dat (NVIDIA Corporation, 2024a).

2.4 Execution model GPU

Způsob, jakým GPU organizuje a spouští výpočty, se výrazně liší od CPU. Pochopení tohoto modelu je základní pro GPU programování.

2.4.1 Vlákna, warpy a bloky

Základní jednotkou výpočtu na GPU je vlákno (thread). Vlákno zpracovává typicky jeden datový prvek a provádí jednu instanci kernelu, tedy funkce, která běží na GPU. GPU spouští tisíce až miliony vláken současně (NVIDIA Corporation, 2024a).

Vlákna jsou organizována do hierarchické struktury:

Vlákno (thread) – nejmenší výpočetní jednotka, zpracovává jeden datový prvek.

Warp – skupina 32 vláken, která se provádějí synchronně (lockstep). Všechna vlákna ve warpu provádějí stejnou instrukci v každém taktu. Warp je skutečná jednotka plánování na GPU.

Blok (thread block) – skupina vláken (typicky 32–1024), která sdílí sdílenou paměť a mohou se synchronizovat pomocí bariér. Jeden blok se provádí na jednom streaming multiprocessoru.

2.4.2 Gridy

Grid je nejvyšší úroveň organizace, tedy logická kolekce bloků, které společně tvoří jednu instanci spuštěného kernelu. Grid může být jednorozměrný, dvourozměrný nebo třírozměrný, podle typu řešeného problému. Například pro zpracování 2D obrázku je přirozené použít 2D grid, kde každý blok zpracovává jeden výřez obrázku (Kirk & Hwu, 2016).

2.4.3 Plánování a divergence

GPU plánovač přiděluje bloky na streaming multiprocessory. Každý SM může mít přiděleno více bloků současně, což pomáhá skrýt latenci přístupu do paměti, zatímco jeden warp čeká na data z paměti, jiný warp může mezitím pracovat (NVIDIA Corporation, 2024a).

Důležitým konceptem je *divergence vláken*. Protože všechna vlákna ve warpu provádějí stejnou instrukci, nastává problém u podmíněného větvení (if/else). Pokud některá vlákna warpu mají vykonat jednu větev a jiná druhou, GPU musí provést obě větve sekvenčně. To probíhá tak, že při zpracovávání instrukcí jedné větve (např. if), jsou vlákna, která mají vykonávat neaktivní větev (např. else) maskována (vyřazena) a čekají, po dokončení instrukcí jedné větve (např. if) se situace obrátí. Po dokončení instrukcí obou větví se oba toky znovu sloučí (reconvergence) a dále běží společně. Tato sekvenční exekuce snižuje výkon a je to jeden z důvodů, proč je na GPU vhodné minimalizovat větvení v kódu (Kirk & Hwu, 2016).

2.5 Přenos dat mezi CPU a GPU

Grafická karta je fyzicky oddělená od hlavního procesoru a komunikuje přes sběrnici PCIe. To znamená, že data je potřeba nejprve přenést z operační paměti (RAM) do paměti GPU (VRAM), provést výpočet a výsledky přenést zpět (Sanders & Kandrot, 2010).

2.5.1 PCIe sběrnice a její limity

Sběrnice PCIe 4.0 x16 nabízí teoretickou propustnost přibližně 32 GB/s v každém směru. PCIe 5.0 zdvojnásobuje tuto hodnotu na 64 GB/s. I tak je to řádově méně než propustnost paměti samotného GPU (například NVIDIA RTX 4090 má paměťovou propustnost okolo 1 TB/s) (Hennessey & Patterson, 2017).

2.5.2 Overhead datového přenosu

Přenos dat mezi CPU a GPU představuje významný overhead, který může u malých úloh zcela zastínit výhodu paralelního výpočtu na GPU. Proto se GPU vyplatí zejména pro úlohy, kde je poměr výpočtu k přenášeným datům dostatečně vysoký (Kirk & Hwu, 2016).

V praxi to znamená, že pro malé matice nebo malé datové sady může být výpočet na CPU rychlejší než na GPU, i přestože samotný výpočet na GPU by byl rychlejší. Existuje takzvaný *crossover point*, tedy velikost vstupu, od které je výpočet na GPU rychlejší než na CPU. Nalezení tohoto bodu patří mezi cíle experimentální části této práce.

Pro minimalizaci přenosů se používá několik technik: pinned (page-locked) memory pro rychlejší přenosy, asynchronní přenosy, které se překrývají s výpočtem, a proudové zpracování (CUDA streams), kde se datový přenos a výpočet provádějí paralelně (NVIDIA Corporation, 2024a).

3 Technologie a nástroje pro GPU výpočty

Po teoretickém úvodu do paralelních výpočtů a architektury GPU se nyní zaměříme na konkrétní nástroje, které GPU programování umožňují. Trh nabízí řadu možností od nízkoúrovňových platforem jako CUDA a OpenCL až po vysokoúrovňové knihovny, které GPU výpočty zpřístupňují programátorům v C# nebo Pythonu. Hlavní pozornost je věnována nástrojům v prostředí .NET, protože praktická část práce je implementována v C#.

3.1 CUDA

CUDA (Compute Unified Device Architecture) je proprietární platforma a programovací model vyvinutý firmou NVIDIA, vydaný poprvé v roce 2007. CUDA umožňuje programovat GPU v rozšíření jazyka C/C++ a je v současnosti nejrozšířenější platformou pro GPU výpočty.

3.1.1 Architektura a programovací model

Programovací model CUDA je založen na konceptu kernelů, tedy funkcí, které se spouštějí paralelně na GPU. Programátor definuje kernel a při jeho spuštění určí, kolik vláken a v jaké konfiguraci (bloky, gridy) se má spustit (Nickolls et al., 2008).

Typický postup práce s CUDA vypadá následovně:

1. Alokace paměti na GPU.
2. Kopírování vstupních dat z CPU paměti do GPU paměti.
3. Spuštění kernelu na GPU.
4. Kopírování výsledků zpět z GPU do CPU paměti.
5. Uvolnění GPU paměti.

CUDA kód se kompiluje pomocí kompilátoru nvcc, který rozdělí kód na host část (běží na CPU) a device část (běží na GPU). Device část se kompiluje do mezikódu PTX (Parallel Thread Execution) a následně do strojového kódu konkrétní GPU architektury (NVIDIA Corporation, 2024a).

3.1.2 Ekosystém a knihovny

Velkou výhodou CUDA je rozsáhlý ekosystém knihoven, které poskytují optimalizované implementace pro běžné výpočetní úlohy (NVIDIA Corporation, 2024b):

- **cuBLAS** – lineární algebra (násobení matic, vektorové operace),
- **cuFFT** – rychlá Fourierova transformace,
- **cuDNN** – operace pro hluboké neuronové sítě,
- **cuRAND** – generování pseudonáhodných čísel,
- **Thrust** – knihovna šablon podobná C++ STL pro GPU.

3.1.3 Výhody a omezení

Největší výhodou CUDA je výkon, zralost ekosystému a kvalitní dokumentace. Nevýhodou je, že CUDA funguje výhradně na GPU od NVIDIA a nelze ji použít na kartách AMD ani Intel. To je zároveň hlavní argument pro alternativní platformy jako OpenCL (Kirk & Hwu, 2016).

3.2 OpenCL

OpenCL (Open Computing Language) je otevřený standard pro paralelní programování na heterogenních platformách, spravovaný konsorciem Khronos Group. Na rozdíl od CUDA je OpenCL navržen jako multiplatformní a podporuje GPU od NVIDIA, AMD i Intel, ale také CPU, FPGA a další akcelerátory (Khronos Group, 2023).

3.2.1 Programovací model

Programovací model OpenCL je konceptuálně podobný CUDA, ale používá trochu odlišnou terminologii. Základní pojmy (Khronos Group, 2024a):

- **Platform** – implementace OpenCL (např. od NVIDIA, AMD, Intel),
- **Device** – konkrétní hardware (GPU, CPU),
- **Context** – prostředí pro správu paměti a spouštění příkazů,
- **Command Queue** – fronta příkazů pro dané zařízení,
- **Kernel** – výpočetní funkce spouštěná na Device,
- **Work-item** – obdoba CUDA vláknů,
- **Work-group** – obdoba CUDA bloku.

Na rozdíl od CUDA se kernely v OpenCL typicky kompilují za běhu (runtime compilation) ze zdrojového kódu v jazyce OpenCL C (podmnožina C99 s rozšířeními). To umožňuje přenositelnost mezi různými zařízeními, ale přidává overhead při startu programu (Scarpino, 2012).

3.2.2 Výhody a omezení

Hlavní výhodou OpenCL je přenositelnost, stejný kód může (s úpravami) běžet na GPU různých výrobců, na CPU i na FPGA. Nevýhodou je, že optimalizace pro konkrétní hardware bývá obtížnější než u CUDA, a ekosystém knihoven a nástrojů je méně rozvinutý. V praxi bývá výkon OpenCL kódu na NVIDIA GPU o něco nižší než u ekvivalentního CUDA kódu (Scarpino, 2012).

3.3 Vulkan Compute

Vulkan je moderní nízkoúrovňové grafické a výpočetní API, taktéž spravované Khronos Group. Vulkan compute shaders umožňují provádět obecné výpočty na GPU pomocí jazyka GLSL nebo HLSL, které se kompilují do mezikódu SPIR-V (Khronos Group, 2024b).

Vulkan je primárně určen pro grafické aplikace a herní enginy, kde nabízí nižší overhead než starší OpenGL. Pro čistě výpočetní úlohy se ale v praxi používá méně než CUDA nebo OpenCL, protože nemá tak rozvinutý ekosystém výpočetních knihoven. V kontextu této práce je zmíněn pro úplnost přehledu technologií.

3.4 Nástroje v prostředí C# / .NET

Přestože CUDA a OpenCL jsou primárně určeny pro C/C++, existuje několik nástrojů, které zpřístupňují GPU výpočty programátorům v prostředí .NET a jazyce C#. Tato sekce podrobně popisuje jednotlivé možnosti, přičemž největší pozornost je věnována knihovně ILGPU, která je hlavním nástrojem praktické části práce.

3.4.1 ILGPU

ILGPU je open-source knihovna, která umožňuje psát GPU výpočty přímo v jazyce C# bez nutnosti znalosti CUDA C/C++ nebo OpenCL C. Autorem je Marcel Koester a knihovna je licencována pod MIT licencí (Koester, 2024a).

3.4.1.1 Architektura a principy

Základní myšlenka ILGPU spočívá v tom, že se C# kód (přesněji IL, Intermediate Language, do kterého se C# kompiluje) za běhu programu kompiluje do GPU kódu. ILGPU obsahuje vlastní kompilátor, který analyzuje IL kód kernelové funkce a překládá ho do PTX (pro CUDA backend) nebo OpenCL kódu (Koester, 2024b).

Programátor píše kernel jako obyčejnou C# metodu, která přijímá speciální index parametr a pracuje s daty v GPU paměti. ILGPU za běhu tuto metodu zkompiluje pro cílový GPU a zajistí její spuštění.

Klíčové třídy a pojmy v ILGPU:

- **Context** – hlavní objekt pro inicializaci ILGPU,
- **Accelerator** – abstrakce nad konkrétním GPU zařízením,
- **MemoryBuffer** – buffer v GPU paměti pro data,
- **Kernel** – zkompilovaný GPU program připravený ke spuštění,
- **Index** – identifikátor vlákna v rámci kernelu.

3.4.1.2 Podporované backendy a multiplatformní podpora

ILGPU podporuje tři backendy (Koester, 2024a):

- **CUDA** – pro NVIDIA GPU, překládá do PTX,
- **OpenCL** – pro AMD a Intel GPU, generuje OpenCL kód,
- **CPU** – vícevláknová softwarová emulace pro ladění a testování.

Díky tomu je kód napsaný v ILGPU přenositelný mezi GPU všech hlavních výrobců bez nutnosti úprav. Stejný C# kernel běží na NVIDIA, AMD i Intel GPU.

Knihovna je plně multiplatformní a podporuje operační systémy Windows, Linux i macOS včetně architektury ARM64. ILGPU je kompatibilní s .NET 5.0 a novějšími.

Součástí ekosystému je doplňková knihovna ILGPU.Algorithms, která poskytuje optimalizované implementace běžných paralelních operací jako řazení, prefix sum (scan), redukce nebo operace s řídkými maticemi.

3.4.1.3 Příklad použití

Jednoduchý kernel pro součet dvou vektorů v ILGPU vypadá následovně:

Ukázka 3.1: Ukázka ILGPU kernelu pro vektorový součet v C#

```
static void VectorAddKernel(
    Index1D index,
    ArrayView<float> a,
    ArrayView<float> b,
    ArrayView<float> result)
{
    result[index] = a[index] + b[index];
}
```

Spuštění kernelu zahrnuje vytvoření kontextu, inicializaci akcelérátoru, alokaci paměťových bufferů, nahrání dat a samotné volání kernelu. Oproti nativní CUDA v C++ je syntaxe čistší a nevyžaduje ruční správu paměti v takové míře.

3.4.2 ComputeSharp

ComputeSharp je knihovna od Sergia Pedri (Microsoft), která umožňuje psát GPU výpočty v C# přes DirectX 12 compute shaders. Programátor implementuje rozhraní IComputeShader a logiku výpočtu zapíše do metody Execute (Pedri, 2024).

Výhodou ComputeSharp je moderní API, dobré napojení na ekosystém Microsoftu a relativně jednoduchá syntaxe. Hlavním omezením je, že podporuje pouze Windows a DirectX 12, nelze ji použít na Linuxu ani macOS a funguje pouze na GPU s podporou DirectX 12 (moderní NVIDIA, AMD i Intel karty). Pro multiplatformní nasazení tedy není vhodná.

3.4.3 Alea GPU

Alea GPU byla komerční knihovna od švýcarské firmy QuantAlea, která umožňovala programovat NVIDIA GPU v jazycích C# a F#. Nabízela jak nízkoúrovňové CUDA API, tak vysokoúrovňové programovací primitiva.

Projekt je v současnosti ukončený, webové stránky (aleagpu.com) jsou offline a doména je nabízena k prodeji. Knihovna je uvedena pro úplnost přehledu, protože se v minulosti jednalo o jednu z prvních možností GPU programování v prostředí .NET. Pro nové projekty je nutné zvolit alternativu, například ILGPU.

3.4.4 Managed CUDA

ManagedCUDA je .NET wrapper pro nativní CUDA Driver API. Na rozdíl od ILGPU, kde se C# kód kompiluje na GPU kód automaticky, ManagedCUDA poskytuje přímý přístup k CUDA API funkcím z C# prostředí. Interně využívá mechanismus P/Invoke (Platform Invocation Services) pro volání nativních funkcí z knihovny CUDA Driver API (Kunz, 2024).

3.4.4.1 Architektura a použití

Typický postup práce s ManagedCUDA vypadá takto:

1. Implementace GPU kernelu v jazyce CUDA C/C++.
2. Kompilace kernelu do PTX kódu pomocí nástroje nvcc.
3. Načtení PTX modulu z C# aplikace pomocí ManagedCUDA.
4. Alokace paměti na GPU a nastavení parametrů kernelu.
5. Spuštění kernelu na GPU.

Výhodou tohoto přístupu je plný přístup ke všem funkcím CUDA API a programátor má stejné možnosti jako při programování v nativním C/C++. Může využít pokročilé funkce jako CUDA streams (asynchronní vykonávání operací), CUDA events (synchronizace a měření času), Dynamic Parallelism (spouštění kernelů z GPU) nebo Cooperative Groups (jemnější synchronizace vláken). (Kunz, 2024).

Nevýhodou je, že kernely se stále píšou v CUDA C/C++, nikoliv v C#. To znamená, že projekt musí obsahovat dva jazyky a programátor potřebuje znalost CUDA C++. Navíc ManagedCUDA podporuje výhradně NVIDIA GPU.

3.4.5 OpenCL .NET wrappery

Obdobně jako ManagedCUDA pro CUDA existují i .NET wrappery pro OpenCL API, které umožňují používat OpenCL z C# kódu.

3.4.5.1 Silk.NET

Silk.NET je moderní open-source knihovna spravovaná v rámci .NET Foundation, která poskytuje nízkoúrovňové vazby (bindings) na řadu grafických a výpočetních API včetně OpenCL, OpenGL, Vulkan a WebGPU (.NET Foundation, 2025). Pro GPU výpočty přes OpenCL nabízí Silk.NET typově bezpečný .NET wrapper, který umožňuje z C# kódu vytvářet OpenCL kontexty, kompilovat kernely, spravovat paměťové buffery a spouštět výpočty na libovolném OpenCL zařízení.

Oproti ManagedCUDA je výhodou podpora GPU od jakéhokoliv výrobce (NVIDIA, AMD, Intel), protože OpenCL je otevřený standard. Knihovna je multiplatformní (Windows, Linux, macOS) a licencována pod MIT licenci. Starší wrappery jako Cloo nebo OpenCL.NET jsou v současnosti neudržované a pro nové projekty se nedoporučují.

3.4.5.2 Srovnání managed wrapperů s nativním přístupem

Managed wrappery (ManagedCUDA, Silk.NET) mají oproti knihovnám jako ILGPU nebo ComputeSharp jednu podstatnou nevýhodu – kernely se nepíší v C#, ale v CUDA C++ respektive OpenCL C. To znamená, že programátor potřebuje znalost dvou jazyků a nemůže využít komfort C# ekosystému při psaní GPU kódu.

Wrappery však nabízejí plný přístup k nativnímu API a tím i k pokročilým optimalizacím, které nemusí být dostupné ve vysokoúrovňových knihovnách. Pro většinu praktických úloh ale vysokoúrovňový přístup (ILGPU) poskytuje dostatečný výkon s výrazně nižším vývojovým úsilím.

3.5 Nástroje v prostředí Python

Python patří v současnosti mezi nejpoužívanější jazyky v oblasti vědeckých výpočtů a strojového učení. Díky tomu existuje i dobrá podpora pro GPU výpočty.

3.5.1 CuPy

CuPy je knihovna, která poskytuje API kompatibilní s NumPy, ale výpočty provádí na GPU. Programátor může často existující NumPy kód přepsat na CuPy pouhou záměnou `import numpy` za `import cupy`. CuPy interně využívá CUDA a od verze 14 podporuje kromě NVIDIA GPU také AMD GPU přes platformu ROCm (Preferred Networks, 2024).

3.5.2 Numba

Numba je JIT (Just-In-Time) kompilátor pro Python, který umí zkompileovat vybrané funkce přímo pro GPU. Programátor označí funkci dekorátorem `@cuda.jit` a Numba ji za běhu zkompileje do CUDA PTX kódu. To umožňuje psát GPU kernely přímo v Pythonu (Anaconda, Inc., 2024).

3.5.3 PyCUDA

PyCUDA je nízkoúrovňový wrapper nad CUDA API pro Python. Kernely se píšou v CUDA C a předávají se jako textové řetězce, které PyCUDA kompiluje za běhu. PyCUDA nabízí plný přístup k CUDA API a je vhodný pro situace, kdy potřebujeme maximální kontrolu (Klöckner, 2024).

3.6 Srovnání technologií

Pro přehlednost shrnuje tabulka 3.1 hlavní vlastnosti představených technologií a nástrojů.

Tabulka č. 3.1: Srovnání technologií pro GPU výpočty

Technologie	Jazyk	GPU	Platforma	Licence	Aktivní
CUDA	C/C++	NVIDIA	Win/Lin	propr.	ano
OpenCL	C (C99)	všechny	Win/Lin/Mac	open	ano
ILGPU	C#	NVIDIA+AMD+Intel	Win/Lin/Mac	MIT	ano
ComputeSharp	C#	DX12 GPU	Windows	MIT	ano
Alea GPU	C#/F#	NVIDIA	Win/Lin	komerční	ukončen
ManagedCUDA	C#+CUDA	NVIDIA	Win/Lin	GPLv3	částečně
Silk.NET	C#+OpenCL	všechny	Win/Lin/Mac	MIT	ano
CuPy	Python	NVIDIA+AMD	Win/Lin	MIT	ano
Numba	Python	NVIDIA	Win/Lin/Mac	BSD	ano
PyCUDA	Python+CUDA	NVIDIA	Win/Lin	MIT	ano

Zdroj: vlastní zpracování

Pro praktickou část této práce byla zvolena knihovna ILGPU. Hlavním důvodem je, že umožňuje psát CPU i GPU kód v jednom jazyce (C#), což zjednodušuje přímé porovnání variant. Díky podpoře CUDA i OpenCL backendů ILGPU zajišťuje přenositelnost mezi GPU různých výrobců. Knihovna je open-source s aktivním vývojem, nevyžaduje znalost CUDA C++ a pro demonstrační účely diplomové práce nabízí dostatečný výkon.

4 Návrh experimentu

Před samotnou implementací je potřeba definovat, co přesně se bude měřit, na jakém hardware a jakým způsobem, aby výsledky byly reprodukovatelné a vypovídající. Následující část popisuje výběr úloh, testovací prostředí a metodiku měření.

4.1 Výběr úloh

Pro demonstraci a porovnání GPU a CPU paralelizace byly vybrány dvě úlohy, které se liší svou povahou, ale obě jsou vhodné pro datový paralelismus.

4.1.1 Násobení matic

Násobení matic je klasická úloha z lineární algebry, která se běžně používá jako benchmark pro měření výkonu paralelních systémů (Kirk & Hwu, 2016). Její výpočetní složitost roste kubicky s rozměrem matice ($O(n^3)$), takže i pro středně velké matice vzniká značná výpočetní zátěž. Každý prvek výsledné matice lze přitom počítat nezávisle na ostatních, což z ní dělá přirozeně paralelizovatelnou úlohu.

Pro násobení dvou matic A o rozměrech $m \times k$ a B o rozměrech $k \times n$ platí, že prvek výsledné matice C na pozici (i, j) je dán vztahem:

$$C_{ij} = \sum_{l=1}^k A_{il} \cdot B_{lj}$$

4.1.2 Game of Life

Game of Life (Conwayova hra života) je buněčný automat navržený matematikem Johnem Conwayem v roce 1970 (Gardner, 1970). Jedná se o dvourozměrnou mřížku buněk, kde každá buňka je buď živá, nebo mrtvá. V každém kroku simulace se stav každé buňky aktualizuje podle počtu jejích živých sousedů:

- Živá buňka s méně než 2 živými sousedy umírá (samota).
- Živá buňka se 2 nebo 3 živými sousedy přežívá.
- Živá buňka s více než 3 živými sousedy umírá (přelidnění).
- Mrtvá buňka s přesně 3 živými sousedy se stane živou (reprodukce).

Game of Life je vhodný pro GPU paralelizaci proto, že každá buňka se aktualizuje nezávisle na základě svého okolí a jedná se o typický datový paralelismus na 2D mřížce. Tato úloha navíc reprezentuje takzvaný *stencil pattern*, tedy vzor, kdy každý prvek závisí na svých sousedech. Stejný vzor se vyskytuje v řadě praktických simulací (proudění tekutin, difuze tepla, zpracování obrazu), takže výsledky jsou přenositelné i na jiné domény.

4.1.3 Zdůvodnění výběru

Kombinace obou úloh pokrývá dva důležité vzory paralelních výpočtů. Násobení matic reprezentuje úlohu z lineární algebry s kubickou výpočetní složitostí a pravidelným přístupem do paměti. Game of Life reprezentuje stencil výpočet, kde je důležitý přístup k sousedním prvkům. Obě úlohy lze snadno škálovat zvětšováním rozměrů vstupních dat.

4.2 Testovací prostředí

4.2.1 Hardwarová konfigurace

Experimenty budou prováděny na počítači s následující konfigurací:

Tabulka č. 4.1: Hardwarová konfigurace testovacího prostředí

Komponent	Specifikace
CPU	AMD Ryzen 5 3600, 6 jader / 12 vláken, 3,6 GHz
RAM	32 GB DDR4
GPU	NVIDIA GeForce RTX 3070, 5888 CUDA jader, 8 GB VRAM

Zdroj: vlastní zpracování

4.2.2 Softwarová konfigurace

Tabulka č. 4.2: Softwarová konfigurace testovacího prostředí

Software	Verze
OS	Microsoft Windows 10 (build 19045)
.NET SDK	10.0.5
ILGPU	1.5.3 (CUDA backend)
ComputeSharp	3.1.0 (DirectX 12 backend)

Zdroj: vlastní zpracování

Pro GPU výpočty budou použity dvě nezávislé knihovny: ILGPU, která generuje CUDA PTX kód a podporuje backendy CUDA i OpenCL, a ComputeSharp, která využívá DirectX 12 compute shaders. Srovnání obou knihoven umožňuje posoudit vliv zvolené GPU abstrakce na výkon.

4.3 Metodika měření

Pro zajištění spolehlivých a reprodukovatelných výsledků je definována následující metodika měření.

4.3.1 Měřené metriky

Pro každou konfiguraci (kombinace úlohy, varianty implementace a velikosti vstupu) budou měřeny:

- **Čas výpočtu** T – doba trvání samotného výpočtu v milisekundách. U GPU varianty zahrnuje i čas přenosu dat (alokace, kopírování na GPU, výpočet, kopírování zpět).
- **Zrychlení** S – poměr $T_{\text{seq}}/T_{\text{par}}$, kde T_{seq} je čas sekvenční CPU implementace.
- **Efektivita** E – zrychlení vydělené počtem procesorů (u CPU varianty).

4.3.2 Velikosti vstupních dat

Pro násobení matic budou testovány čtvercové matice následujících rozměrů: 64^2 , 128^2 , 256^2 , 512^2 , 1024^2 , 2048^2 a 4096^2 .

Pro Game of Life budou testovány čtvercové mřížky rozměrů: 8^2 , 12^2 , 16^2 , 24^2 , 32^2 , 64^2 , 100^2 , 150^2 , 250^2 , 500^2 , 1000^2 , 2000^2 , 5000^2 a $10\,000^2$. Menší mřížky (8^2 až 64^2) jsou zahrnuty pro přesné určení crossover pointu, tedy velikosti, od které se GPU vyplatí. Počet simulačních kroků bude 100.

4.3.3 Konfigurace paralelismu

U CPU paralelní varianty bude testován následující počet vláken: 1, 2, 4, 6, 8 a 12. Výběr těchto hodnot odpovídá architektuře použitého procesoru (6 fyzických jader, 12 logických vláken s hyperthreadingem). Testy pro 1, 2, 4 a 6 vláken pokrývají škálování na fyzických jádrech; testy pro 8 a 12 vláken pak zkoumají, jak efektivně hyperthreading dokáže přispět k dalšímu zrychlení. Počet vláken bude řízen pomocí vlastnosti `MaxDegreeOfParallelism` třídy `ParallelOptions` v C#.

U GPU varianty je konfigurace velikosti bloku vláken ponechána na automatickém nastavení knihovny `ILGPU` (`LoadAutoGroupedStreamKernel`), která zvolí optimální velikost bloku na základě parametrů konkrétního GPU. `ComputeSharp` obdobně konfiguruje dispatch automaticky.

4.3.4 Statistická robustnost

Každé měření bude provedeno podle následujícího postupu:

1. **Warmup** – 3 běhy, jejichž výsledky se zahazují. Účelem je zajistit, aby JIT kompilátor .NET provedl optimalizovanou rekompilaci měřeného kódu (přechod z Tier 0 na Tier 1 v rámci tiered compilation) a aby se inicializoval GPU kontext včetně JIT kompilace PTX kódu na strojový kód GPU. Bez warmupu by první měření zahrnovalo jednorázový overhead desítek až stovek milisekund.
2. **Probe run** – jeden zkušební běh po warmupu, na jehož základě se adaptivně upraví počet měřených běhů: pokud probe run trvá déle než 60 s, provedou se pouze 3 měřené běhy; pokud déle než 30 s, provedou se 5 měřených běhů; jinak se provede standardních 10 běhů.
3. **Měření** – 3 až 10 běhů (podle výsledku probe run), z nichž se vypočítá medián a relativní směrodatná odchylka (CV) v procentech. Medián je odolnější vůči odlehlým hodnotám než průměr.

Variabilita běhů je reportována jako relativní směrodatná odchylka (koeficient variace, CV) v procentech, definovaná jako podíl směrodatné odchylky a aritmetického průměru. Pro naměřené časy $x_1, \dots, x_n$ platí:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i \quad (4.1)$$

$$SD = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2} \quad (4.2)$$

$$CV [\%] = \frac{SD}{\bar{x}} \cdot 100. \quad (4.3)$$

4.3.5 Weak-scaling experiment

Pro ověření Gustafsonova zákona v praxi byl proveden weak-scaling experiment, ve kterém se velikost úlohy škáluje s počtem procesorů tak, aby práce na vlákno zůstávala přibližně konstantní. Základní velikost (base) byla zvolena 512; pro každý počet vláken P se velikost vstupu škáluje podle vzorce specifického pro danou úlohu.

Volba škálování vychází z asymptotické výpočetní složitosti jednotlivých algoritmů. U klasického násobení matic velikosti $n \times n$ je počet operací úměrný $O(n^3)$. Aby při weak scalingu zůstala práce na jedno vlákno přibližně konstantní, musí platit $n^3/P \approx \text{konst.}$, z čehož vyplývá $n \sim P^{1/3}$. U Game of Life je pro jednu iteraci na mřížce $n \times n$ množství práce úměrné počtu buněk, tedy $O(n^2)$. Analogicky tedy platí $n^2/P \approx \text{konst.}$, a proto $n \sim \sqrt{P}$.

- **Násobení matic:** $n = \text{base} \cdot P^{1/3} = 512 \cdot P^{1/3}$
- **Game of Life:** $n = \text{base} \cdot \sqrt{P} = 512 \cdot \sqrt{P}$

Tabulka 4.3 uvádí škálované velikosti pro testované počty vláken.

Tabulka č. 4.3: Škálované velikosti vstupů pro weak-scaling experiment

Počet vláken	MatMul n	GoL n
2	645	724
4	813	1024
6	930	1254
8	1024	1448
12	1172	1774

Zdroj: vlastní zpracování

Pro každou kombinaci velikosti a počtu vláken bylo provedeno 10 běhů a vypočítány statistiky (medián, průměr, směrodatná odchylka, koeficient variace). Výsledky umožňují přímé porovnání naměřeného zrychlení s Gustafsonovým teoretickým modelem $S_G(N) = N - \alpha(N - 1)$.

4.4 Očekávané výstupy

Na základě naměřených dat budou vytvořeny následující vizualizace a analýzy:

- Graf závislosti zrychlení na počtu CPU vláken pro různé velikosti vstupu.
- Graf porovnání časů výpočtu CPU sekvenční, CPU paralelní a GPU varianty v závislosti na velikosti vstupu.
- Identifikace crossover pointu - velikosti vstupu, od které GPU začíná být rychlejší než CPU.
- Porovnání naměřeného zrychlení CPU paralelní varianty s teoretickým Amdahlovým zákonem (odhad podílu paralelizovatelné části P z dat).
- Weak-scaling experiment pro validaci Gustafsonova zákona se škálovanými velikostmi vstupů dle počtu vláken.
- Srovnání výkonu dvou GPU knihoven: ILGPU (CUDA) versus ComputeSharp (DirectX 12).
- Analýza overheadu GPU inicializace a přenosu dat na základě chování pro malé vstupy.

5 Implementace

S definovaným návrhem experimentu lze přistoupit k samotné implementaci. Všechny varianty obou úloh (sekvenční CPU, paralelní CPU, GPU přes ILGPU i GPU přes ComputeSharp) jsou napsány v C# s cílovým frameworkem .NET 10. Dále je popsána architektura řešení, ukázky kódu a benchmark framework.

5.1 Architektura řešení

Celé řešení je implementováno jako konzolová aplikace v .NET 10 s balíčky ILGPU a ComputeSharp (nainstalovanými přes NuGet). Projekt je strukturován tak, aby bylo snadné přidávat další úlohy a varianty implementací.

5.1.1 Struktura projektu

Projekt obsahuje následující hlavní třídy a rozhraní:

IBenchmark – rozhraní definující metody `Run()`, `Validate()` a `GetResult()` pro každou variantu implementace.

BenchmarkRunner – třída odpovědná za spouštění benchmarků, měření času, warmup a export výsledků.

GpuSelector – třída pro výběr GPU zařízení (podpora CLI argumentů, proměnných prostředí i interaktivního výběru).

MatrixMultiply* – implementace násobení matic v sekvenční, paralelní CPU, GPU ILGPU a GPU ComputeSharp variantě.

GameOfLife* – implementace Game of Life v sekvenční, paralelní CPU, GPU ILGPU a GPU ComputeSharp variantě.

5.1.2 Rozhraní pro benchmark

Každá varianta implementace realizuje rozhraní `IBenchmark`, které zajišťuje konzistentní způsob spuštění a měření:

Ukázka 5.1: Rozhraní IBenchmark

```
public interface IBenchmark
{
    string Name { get; }
    void Setup(int size);
    void Run();
    bool Validate(IBenchmark reference);
    void Cleanup();
    float[] GetResult();
}
```

Třída BenchmarkRunner poté pro každou implementaci volá metody v pořadí: Setup (příprava dat), warmup běhy, měřené běhy s ukládáním časů, Validate (ověření správnosti výsledku oproti referenční sekvenční implementaci) a Cleanup (uvolnění paměti). Metoda GetResult() vrací výsledek jako pole float [], což umožňuje porovnání výstupů různých variant.

5.2 Násobení matic

5.2.1 CPU sekvenční implementace

Sekvenční verze násobení matic používá tři vnořené smyčky, klasický algoritmus se složitostí $O(n^3)$:

Ukázka 5.2: Sekvenční násobení matic v C#

```
public void Run()
{
    for (int i = 0; i < size; i++)
    {
        for (int j = 0; j < size; j++)
        {
            float sum = 0;
            for (int k = 0; k < size; k++)
            {
                sum += a[i * size + k] * b[k * size + j];
            }
            result[i * size + j] = sum;
        }
    }
}
```

Matice jsou uloženy jako jednorozměrná pole (`float []`) v row-major pořadí, což je běžný přístup pro práci s GPU, kde se jednorozměrná pole spravují snáze.

5.2.2 CPU paralelní implementace

Paralelní verze využívá třídu `Parallel.For` z knihovny TPL (Task Parallel Library), která je součástí .NET. Vnější smyčka (přes řádky matice) je paralelizována, vnitřní dvě smyčky zůstávají sekvenční:

Ukázka 5.3: Paralelní násobení matic na CPU v C#

```
public void Run()
{
    Parallel.For(0, size, new ParallelOptions
    {
        MaxDegreeOfParallelism = threadCount
    }, i =>
    {
        for (int j = 0; j < size; j++)
        {
            float sum = 0;
            for (int k = 0; k < size; k++)
            {
                sum += a[i * size + k] * b[k * size + j];
            }
            result[i * size + j] = sum;
        }
    });
}
```

Parametr `threadCount` umožňuje řídit počet použitých vláken, což je potřebné pro měření závislosti zrychlení na počtu procesorů.

5.2.3 GPU implementace v ILGPU

GPU verze využívá ILGPU pro kompilaci a spuštění kernelu na grafické kartě. Kernel počítá jeden prvek výsledné matice:

Ukázka 5.4: GPU kernel pro násobení matic v ILGPU

```
static void MatrixMultiplyKernel(  
    Index2D index,  
    ArrayView2D<float, Stride2D.DenseX> a,  
    ArrayView2D<float, Stride2D.DenseX> b,  
    ArrayView2D<float, Stride2D.DenseX> result,  
    int size)  
{  
    int row = index.X;  
    int col = index.Y;  
    float sum = 0;  
    for (int k = 0; k < size; k++)  
    {  
        sum += a[row, k] * b[k, col];  
    }  
    result[row, col] = sum;  
}
```

Spuštění kernelu zahrnuje inicializaci ILGPU kontextu, vytvoření akcelérátoru, alokaci GPU bufferů, kopírování dat na GPU, volání kernelu a kopírování výsledků zpět. Kernel je načten pomocí metody `LoadAutoGroupedStreamKernel`, která automaticky zvolí optimální velikost bloku vláken. Po volání kernelu se provede synchronizace a výsledky se zkopírují zpět do paměti CPU. Kompletní kód spuštění je uveden v příloze, viz GPU násobení matic pomocí ILGPU (A.4).

5.2.4 Poznámka k optimalizaci

V rámci této práce je použit naivní GPU kernel, který počítá každý prvek výsledné matice nezávisle a přistupuje přímo do globální paměti. Optimalizace pomocí sdílené paměti (tiling), která je podrobně popsána v Kirk a Hwu (2016), není implementována. Cílem experimentu je primárně porovnat CPU a GPU přístup, nikoli optimalizovat GPU kernel na maximum. I s naivní implementací lze očekávat výrazné zrychlení pro dostatečně velké matice, protože GPU disponuje tisíci jader pracujících paralelně. Velikost bloku vláken je ponechána na automatickém nastavení ILGPU (LoadAutoGroupedStreamKernel).

Stejný naivní algoritmus se třemi vnořenými smyčkami je záměrně použit ve všech variantách implementace, včetně sekvenční a paralelní CPU verze. Varianty se liší pouze způsobem distribuce práce (sekvenční smyčka, `Parallel.For`, GPU kernel), nikoliv použitým algoritmem. Tím je zajištěno, že naměřené rozdíly ve výkonu odrážejí vliv architektury (CPU vs. GPU) a nikoliv vliv odlišného algoritmu či optimalizace.

5.3 Game of Life

5.3.1 CPU sekvenční implementace

Sekvenční implementace prochází celou mřížku a pro každou buňku počítá živé sousedy. Používá se technika double-buffering, tedy výpočet čte z jednoho pole a zapisuje do druhého, aby nedocházelo ke konfliktu čtení a zápisu. Struktura kódu odpovídá stejnému vzoru jako u násobení matic: dvě vnořené smyčky procházejí mřížku, pro každou buňku se spočítá počet živých sousedů a na základě pravidel se určí nový stav. Kompletní zdrojový kód je uveden v příloze, viz Sekvenční Game of Life (A.5).

5.3.2 CPU paralelní implementace

Paralelní verze na CPU opět používá `Parallel.For` pro vnější smyčku přes řádky mřížky. Každý řádek se zpracovává nezávisle, takže nevznikají konflikty při zápisu. Technika double-buffering s prohazováním polí po každém kroku zajišťuje správnost. Paralelizace je provedena stejným způsobem jako u násobení matic, tedy obalením vnější smyčky do `Parallel.For` s nastavitelným `MaxDegreeOfParallelism`. Kompletní zdrojový kód je uveden v příloze, viz Paralelní CPU Game of Life (A.6).

5.3.3 GPU implementace v ILGPU

GPU kernel pro Game of Life zpracovává jednu buňku mřížky. Každé vlákno přečte svou buňku a 8 sousedních buněk z globální paměti, spočítá nový stav a zapíše ho do výstupního pole. Používá se technika ping-pong double buffering, tedy dva buffery se střídají jako vstupní a výstupní v každém kroku simulace:

Ukázka 5.5: GPU kernel pro Game of Life v ILGPU

```
static void GoLKernel(
    Index1D index,
    ArrayView1D<int, Stride1D.Dense> current,
    ArrayView1D<int, Stride1D.Dense> next,
    int n)
{
    int y = index / n;
    int x = index % n;
    int neighbors = 0;
    for (int dy = -1; dy <= 1; dy++)
        for (int dx = -1; dx <= 1; dx++)
        {
            if (dx == 0 && dy == 0) continue;
            int nx = (x + dx + n) % n;
            int ny = (y + dy + n) % n;
            neighbors += current[ny * n + nx];
        }
    bool alive = current[y * n + x] == 1;
    next[y * n + x] = alive
        ? ((neighbors == 2 || neighbors == 3) ? 1 : 0)
        : (neighbors == 3 ? 1 : 0);
}
```

Spuštění kernelu probíhá ve smyčce přes 100 kroků, kde se po každém kroku prohodí vstupní a výstupní buffer:

Ukázka 5.6: Ping-pong double buffering na GPU

```
for (int step = 0; step < 100; step++)
{
    _kernel!(total, current.View, next.View, n);
    accelerator.Synchronize();
    (current, next) = (next, current);
}
current.CopyToCPU(_result!);
```

5.4 Měřicí framework

5.4.1 Třída BenchmarkRunner

Třída BenchmarkRunner zapouzdřuje logiku měření. Pro každou konfiguraci (implementace + velikost vstupu) provede warmup, naměří zadaný počet běhů, vypočítá statistiky (medián, průměr a relativní směrodatná odchylka (CV)) a výsledky uloží.

5.4.2 Výstup dat – CSV formát

Výsledky měření se ukládají do CSV souboru s následujícími sloupci: název implementace, velikost vstupu, číslo běhu, čas v milisekundách. Tato data jsou následně importována a vizualizována v kapitole Výsledky měření.

5.5 GPU implementace v ComputeSharp

Implementace v ComputeSharp (popsané v sekci 3.4.2) využívá odlišný přístup k definici GPU kernelů. Kernel se definuje jako struktura implementující rozhraní IComputeShader s metodou Execute(), která se volá pro každé vlákno. Buffery se předávají jako readonly/readwrite parametry struktury:

Ukázka 5.7: ComputeSharp kernel pro násobení matic

```
[ThreadGroupSize(DefaultThreadGroupSizes.X)]
[GeneratedComputeShaderDescriptor]
internal readonly partial struct MatMulShader : IComputeShader
{
    public readonly ReadOnlyBuffer<float> A;
    public readonly ReadOnlyBuffer<float> B;
    public readonly ReadWriteBuffer<float> C;
    public readonly int N;

    public void Execute()
    {
        int index = ThreadIds.X;
        int row = index / N;
        int col = index % N;
        float sum = 0f;
        for (int k = 0; k < N; k++)
            sum += A[row * N + k] * B[k * N + col];
        C[row * N + col] = sum;
    }
}
```

Spuštění kernelu je jednodušší než u ILGPU, stačí zavolat `device.For()` s počtem vláken a instancí shaderu:

Ukázka 5.8: Spuštění ComputeSharp kernelu

```
var device = GraphicsDevice.GetDefault();
using var bufA = device.AllocateReadOnlyBuffer<float>(a);
using var bufB = device.AllocateReadOnlyBuffer<float>(b);
using var bufC = device.AllocateReadWriteBuffer<float>(n * n);
device.For(n * n, new MatMulShader(bufA, bufB, bufC, n));
bufC.CopyTo(result);
```

Atribut `[GeneratedComputeShaderDescriptor]` zajistí automatické generování HLSL kódu v době kompilace pomocí source generatorů, takže není potřeba ručně psát HLSL.

5.5.1 Omezení 1D dispatche a přechod na 2D

Při použití ComputeSharp došlo k chybě způsobené překročením limitu DirectX 12 pro maximální počet thread groups v jedné dimenzi. DirectX 12 povoluje nejvýše 65 535 thread groups na dimenzi, takže při 1D dispatchi s velkým počtem vláken (například násobení matic 2048^2 vyžaduje přes 4 miliony vláken) hodnota překročila tento limit a program selhal.

Řešením bylo použití 2D dispatche, který rozložil výpočet do dvou dimenzí. Místo volání `device.For(n * n, shader)` se používá `device.For(n, n, shader)`, čímž se počet thread groups v každé dimenzi sníží na přijatelnou hodnotu. Po této úpravě ComputeSharp funguje pro všechny testované velikosti vstupů.

6 Výsledky měření

Naměřená data jsou pro každou úlohu prezentována formou tabulek s absolutními časy, zrychlením oproti sekvenční variantě a porovnáním s teoretickými modely. U CPU paralelní varianty jsou měřeny konfigurace s 1, 2, 4, 6, 8 a 12 vláken. V tabulkách je uvedena varianta s maximálním počtem vláken, kompletní data jsou v příloze. U GPU variant jsou porovnány knihovny ILGPU a ComputeSharp v jejich automatických konfiguracích. Hodnoty odpovídají mediánům z měřených běhů po třech zahřívacích bězích (počet měřených běhů byl adaptivně 3 až 10 podle délky výpočtu). Všechny experimenty byly provedeny na sestavě popsané v kapitola 4 (AMD Ryzen 5 3600, NVIDIA RTX 3070). Relativní směrodatné odchylky časů (koeficient variace, *CV*) jsou uvedeny u tabulek s absolutními časy a kompletní statistiky pro všechny konfigurace jsou uvedeny v příloze kapitola B.

6.1 Násobení matic

6.1.1 Absolutní časy výpočtu

Tabulka 6.1 uvádí naměřené mediány časů výpočtu násobení matic pro čtyři hlavní varianty implementací a sedm velikostí vstupních matic.

Tabulka č. 6.1: Časy výpočtu násobení matic [ms] – medián

Rozměr	CPU seq.	CPU par. (max)	ILGPU	ComputeSharp
64^2	0,19	0,19	3,04	2,20
128^2	2,31	0,61	3,18	2,39
256^2	17,98	4,15	3,28	3,05
512^2	186,68	36,90	4,11	3,60
1024^2	5274,57	359,35	8,18	29,32
2048^2	69 106,42	7674,96	34,59	49,49
4096^2	1 463 460,11	144 040,86	640,33	223,96

Zdroj: vlastní zpracování

Tabulka č. 6.2: Časy výpočtu násobení matic – relativní směrodatná odchylka (CV) [%]

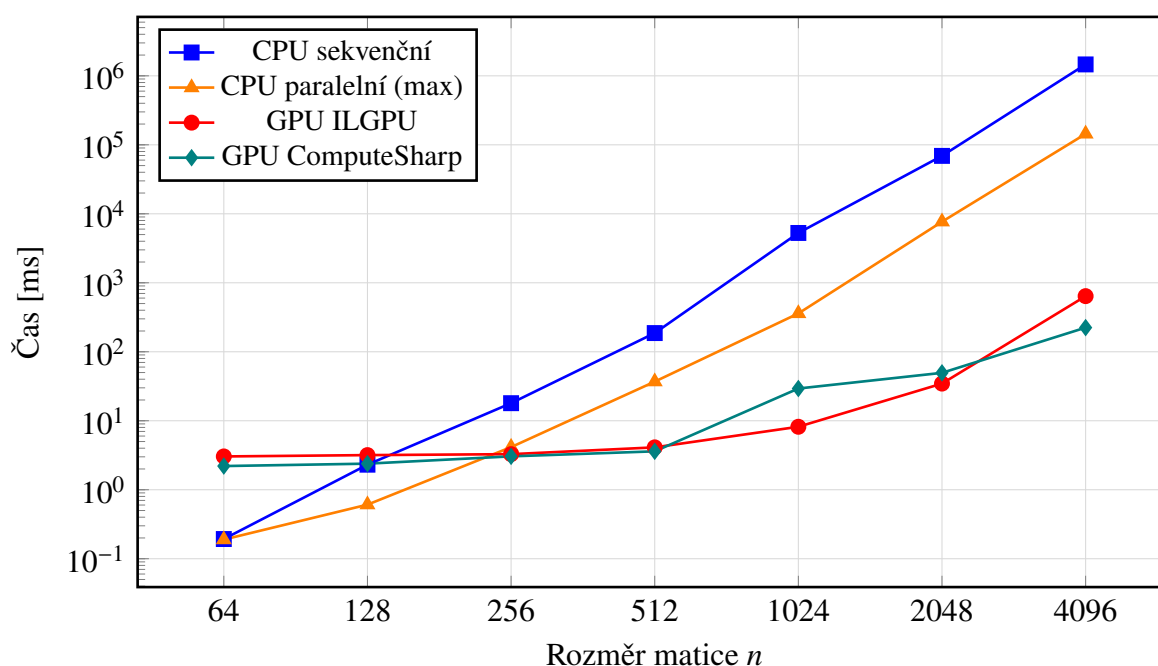
Rozměr	CPU seq.	CPU par. (max)	ILGPU	ComputeSharp
64^2	1,01	9,26	2,56	3,66
128^2	0,78	10,51	7,22	6,91
256^2	7,71	8,31	2,08	7,96
512^2	1,40	3,21	3,57	6,51
1024^2	3,59	3,62	5,61	3,65
2048^2	21,93	50,74	11,36	7,36
4096^2	8,38	15,34	0,83	4,31

Zdroj: vlastní zpracování

Kompletní výpis relativních směrodatných odchylek pro násobení matic je uveden v příloze v tabulkách tabulky B.2 and B.4.

Hodnoty ComputeSharp pro matice 2048^2 a 4096^2 byly naměřeny po přechodu na 2D dispatch (viz sekce 5.5), který vyřešil omezení DirectX 12 pro počet thread groups při 1D dispatchi.

Na obrázek 6.1 jsou časy výpočtu zobrazeny v log-log grafu, který umožňuje přehledně porovnat všechny varianty napříč šesti řády velikostí (od 0,19 ms po více než 600 s).

Obrázek č. 6.1: Časy výpočtu násobení matic – porovnání variant (log-log škála)**Zdroj: vlastní zpracování**

Z grafu je patrné, že GPU varianta (ILGPU) se stává rychlejší než sekvenční CPU mezi velikostmi 128^2 a 256^2 . Pro matici 256^2 je GPU již $5,5\times$ rychlejší než sekvenční CPU a srovnatelné s CPU paralelní variantou (3,28 ms vs 4,15 ms). Od velikosti 512^2 GPU dominuje všem CPU variantám.

6.1.2 Škálování CPU paralelní varianty

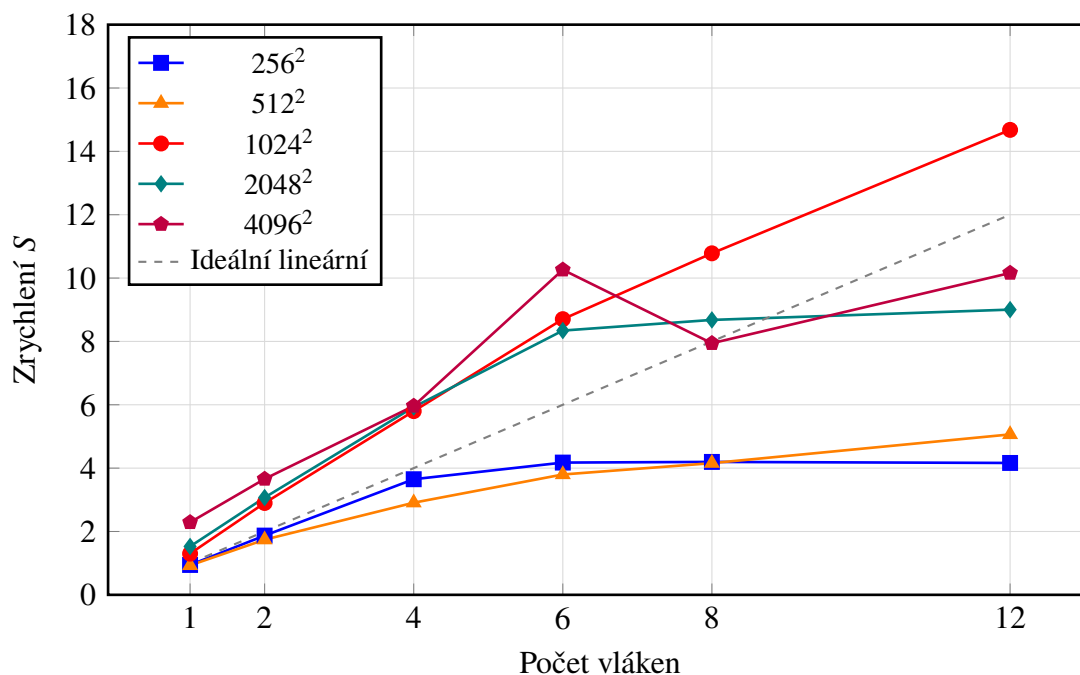
Tabulka 6.3 ukazuje závislost zrychlení CPU paralelní varianty na počtu vláken pro vybrané velikosti matic. Zrychlení CPU paralelní varianty je definováno jako poměr času sekvenční CPU varianty a času paralelní CPU varianty pro danou velikost matice.

Tabulka č. 6.3: Zrychlení CPU paralelní varianty – násobení matic

Vlákna	256^2	512^2	1024^2	2048^2	4096^2
1	0,98	0,93	1,30	1,53	2,29
2	1,95	1,74	2,90	3,07	3,66
4	3,80	2,83	5,80	5,93	5,96
6	4,35	3,67	8,70	8,34	10,26
8	4,37	4,15	10,78	8,68	7,94
12 (max)	4,34	5,06	14,68	9,00	10,16

Zdroj: vlastní zpracování

Obrázek č. 6.2: Zrychlení CPU paralelní varianty vs. počet vláken – násobení matic



Zdroj: vlastní zpracování

Matice 1024^2 vykazuje super-lineární zrychlení ($14,7\times$ na 12 vláknech oproti sekvenční variantě). Část tohoto efektu by mohla být způsobena rozdílem v JIT kompilaci: paralelní varianta s jedním vláknem je sama o sobě $1,3\times$ rychlejší než sekvenční.

Příčinou super-lineárního škálování je neobvykle pomalý běh sekvenční varianty; přesný důvod tohoto zpomalení nelze bez profilování s hardware performance counters určit s jistotou. Skutečnost, že i Parallel(1) na jednom jádře je $1,3\times$ rychlejší, naznačuje, že příčina nesouvisí pouze s počtem jader, ale pravděpodobně i s rozdíly v generovaném JIT kódu či přístupových vzorech paralelního frameworku. I po normalizaci na Parallel(1) však zůstává zrychlení super-lineární na vláknech 2–8, což ukazuje na další faktor související s víceprocesorovým zpracováním, jehož přesnou povahu bez profilování nelze určit (podrobněji viz sekce 7.4).

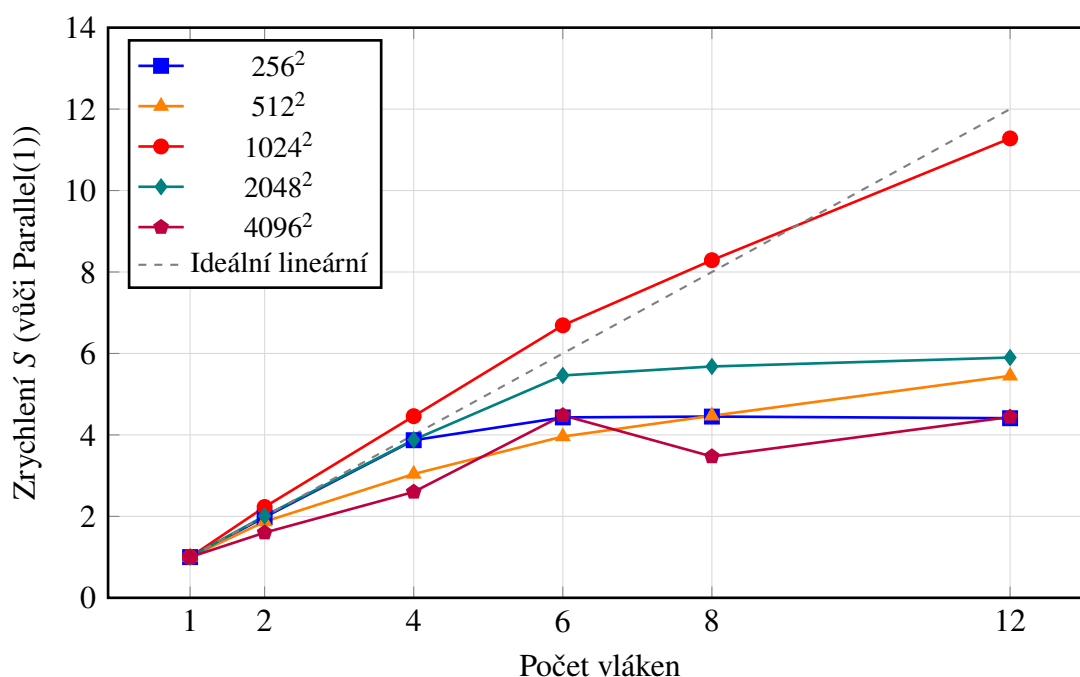
Pro oddělení vlivu JIT kompilace od efektu paralelizace uvádí tabulka 6.4 a obrázek 6.3 zrychlení normalizované na Parallel(1). Vzhledem k popsání anomálie sekvenční varianty bude nadále v rámci Amdahlovy analýzy násobení matic odvozeno právě z Parallel(1), čímž se odstraní zkreslení jednojádrového sekvenčního běhu. U Game of Life zůstává referencí sekvenční varianta, jejíž čas se od Parallel(1) liší o méně než půl procenta.

Tabulka č. 6.4: Zrychlení CPU paralelní varianty normalizované na Parallel(1) – násobení matic

Vlákna	256 ²	512 ²	1024 ²	2048 ²	4096 ²
1	1,00	1,00	1,00	1,00	1,00
2	1,98	1,87	2,23	2,01	1,60
4	3,87	3,04	4,46	3,88	2,60
6	4,43	3,96	6,69	5,46	4,48
8	4,45	4,47	8,29	5,68	3,47
12	4,41	5,45	11,28	5,90	4,44

Zdroj: vlastní zpracování

Obrázek č. 6.3: Zrychlení CPU paralelní varianty normalizované na Parallel(1) – násobení matic



Zdroj: vlastní zpracování

I po normalizaci zůstává zrychlení matice 1024² super-lineární na všech vláknech bez hyperthreadingu (2–8), sub-lineární je až při 12 vláknech (11,28×). To vylučuje, že by šlo pouze o artefakt porovnání odlišných implementací; přesná příčina přetrvávajícího super-lineárního škálování na fyzických jádrech zůstává bez profilování neznámá (viz sekce 7.4). Pro ostatní velikosti (2048², 4096²) je normalizované zrychlení sub-lineární a odpovídá reálnému počtu fyzických jader.

Zajímavostí u matice 4096^2 je pokles efektivity z $4,48\times$ na 6 vláknech na $3,47\times$ při 8 vláknech a následný návrat na $4,44\times$ při 12 vláknech. Tento pokles při přechodu z fyzických jader na hyperthreading naznačuje, že výpočet je omezen propustností paměťového řadiče při velkém objemu dat (přibližně 192 MB pro tři matice).

6.1.3 GPU zrychlení

Tabulka 6.5 uvádí zrychlení GPU variant oproti sekvenční CPU implementaci.

Tabulka č. 6.5: Zrychlení GPU variant oproti sekvenční CPU – násobení matic

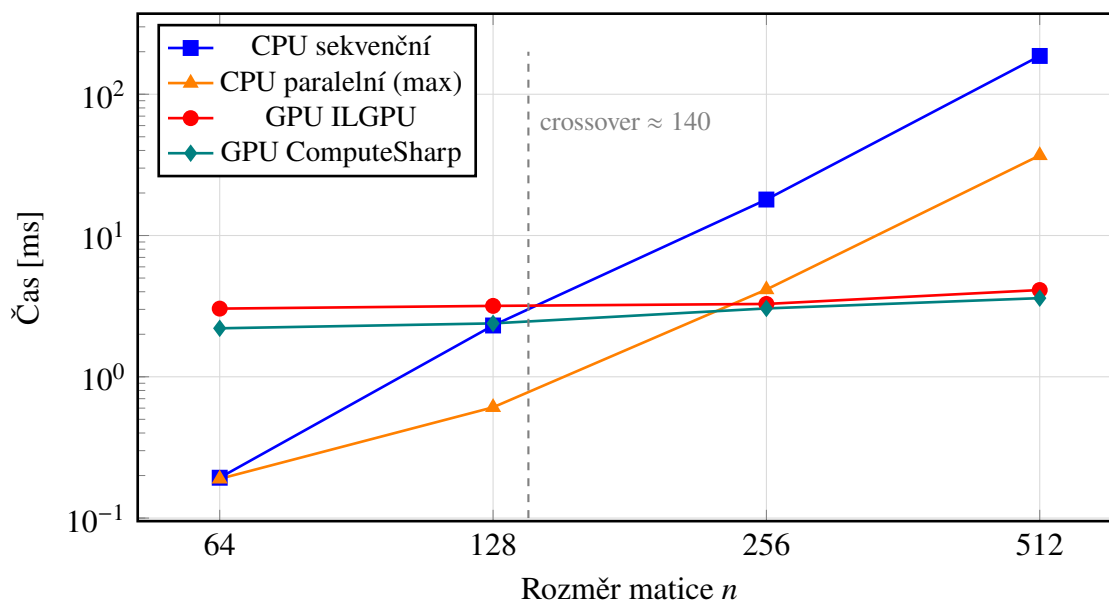
Rozměr	ILGPU	ComputeSharp
64^2	0,06	0,09
128^2	0,73	0,97
256^2	5,48	5,90
512^2	45,39	51,81
1024^2	644,69	179,90
2048^2	1997,94	1396,46
4096^2	2285,47	6534,36

Zdroj: vlastní zpracování

GPU zrychlení roste dramaticky s velikostí matice: od $0,06\times$ (GPU pomalejší) pro 64^2 po $2285\times$ pro 4096^2 (ILGPU). Zajímavým poznatkem je, že ComputeSharp je pro největší matice 4096^2 výrazně rychlejší než ILGPU ($6534\times$ vs $2285\times$), což může být způsobeno efektivnější správou paměti v DirectX 12 backendu pro velké datové sady.

6.1.4 Crossover point – CPU versus GPU

Obrázek č. 6.4: Crossover point – násobení matic (detail pro malé velikosti)



Zdroj: vlastní zpracování

Z detailního pohledu na oblast malých matic (obrázek 6.4) vyplývá, že GPU ILGPU se stává rychlejší než sekvenční CPU přibližně pro matice velikosti 140×140 . GPU se vyrovná CPU paralelní variantě (12 vláken) kolem velikosti 256^2 .

Příčinou je, že GPU overhead (inicializace kontextu, alokace bufferů, kopírování dat, spuštění kernelu) činí přibližně 3 ms bez ohledu na velikost vstupu. Pro matice 64^2 je samotný výpočet na CPU rychlejší (0,19 ms), takže GPU overhead dominuje. Se zvětšující se maticí roste výpočetní čas kubicky ($O(n^3)$), zatímco objem přenášených dat roste kvadraticky ($O(n^2)$), díky čemuž GPU rychle získává převahu.

6.2 Game of Life

6.2.1 Absolutní časy výpočtu

Tabulka 6.6 uvádí naměřené časy pro 100 kroků simulace Game of Life.

Tabulka č. 6.6: Časy výpočtu Game of Life (100 kroků) [ms] – medián

Rozměr mřížky	CPU seq.	CPU par. (max)	ILGPU	ComputeSharp
8^2	0,38	0,70	4,34	9,00
32^2	6,16	1,89	4,42	8,87
100^2	59,45	13,11	4,34	8,40
500^2	1438,08	257,54	5,26	16,85
1000^2	5750,23	971,27	8,36	50,54
2000^2	23 020,42	3871,23	22,55	38,27
5000^2	143 819,33	23 919,09	98,47	166,63
$10\,000^2$	575 307,01	95 332,62	368,47	788,81

Zdroj: vlastní zpracování

Tabulka č. 6.7: Časy výpočtu Game of Life (100 kroků) – relativní směrodatná odchylka (CV) [%]

Rozměr mřížky	CPU seq.	CPU par. (max)	ILGPU	ComputeSharp
8^2	0,55	3,05	4,23	1,15
32^2	3,99	1,93	2,48	3,20
100^2	0,82	2,58	21,24	2,05
500^2	0,12	0,18	15,15	2,49
1000^2	0,03	0,06	17,86	31,19
2000^2	0,04	0,05	7,80	96,95
5000^2	0,01	0,04	2,44	2,46
$10\,000^2$	0,01	0,02	1,10	7,00

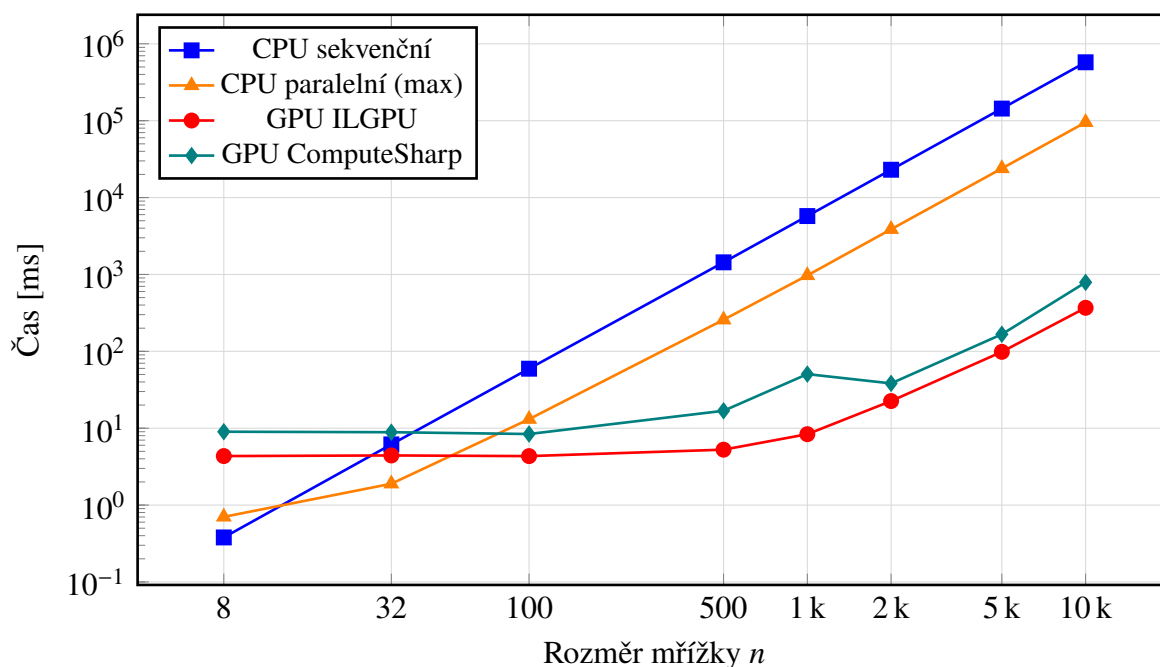
Zdroj: vlastní zpracování

Z tabulky je patrná výrazně vyšší variabilita GPU měření ve srovnání s CPU implementacemi. CPU varianty vykazují stabilní hodnoty s CV pod 4 %, zatímco GPU varianty dosahují CV až 96,95 % (ComputeSharp pro 2000^2) a 21,24 % (ILGPU pro 100^2). Tyto výsledky je nutné interpretovat s opatrností, protože takto vysoká variabilita snižuje jejich věrohodnost. Přesné příčiny nejsou známy bez detailního profilování, možnými faktory mohou být variabilita plánování GPU,

vlivy teplotního managementu či kolísání režie ovladače. Zajímavostí je, že pro jiné velikosti jako 5000^2 dosahuje stejná implementace ComputeSharp stabilních hodnot ($CV = 2,46\%$), což naznačuje specifickou citlivost na určité velikosti vstupů.

Kompletní výpis relativních směrodatných odchylek pro Game of Life je uveden v příloze v tabulce tabulka B.6.

Obrázek č. 6.5: Časy výpočtu Game of Life – porovnání variant (log-log škála)



Zdroj: vlastní zpracování

Na rozdíl od násobení matic je GPU varianta rychlejší než sekvenční CPU **pro velikosti od 32^2 výše**. Pro mřížku 8^2 je GPU pomalejší než CPU, ale již pro 32^2 je ILGPU rychlejší (4,42 ms vs 6,16 ms sekvenčně). Důvodem je, že Game of Life provádí 100 iterací, přičemž výpočetní složitost jedné iterace je $O(n^2)$, celkem tedy $O(100 \cdot n^2)$. Objem přenášených dat je přitom pouze $O(n^2)$. Poměr výpočtu k přenosu je zde tedy přibližně 100× vyšší než u jednoho kroku, takže i pro malou mřížku dominuje čas výpočtu nad GPU overheadem.

Pro nalezení crossover pointu byly dodatečně otestovány menší mřížky. GPU (ILGPU) se stává pomalejší než sekvenční CPU pro mřížky o straně menší než přibližně 28. Pro mřížku 24^2 je GPU pomalejší (4,41 ms vs 3,34 ms), zatímco pro 32^2 je GPU rychlejší (4,42 ms vs 6,16 ms). Crossover point u Game of Life leží tedy přibližně mezi mřížkami 24^2 a 32^2 .

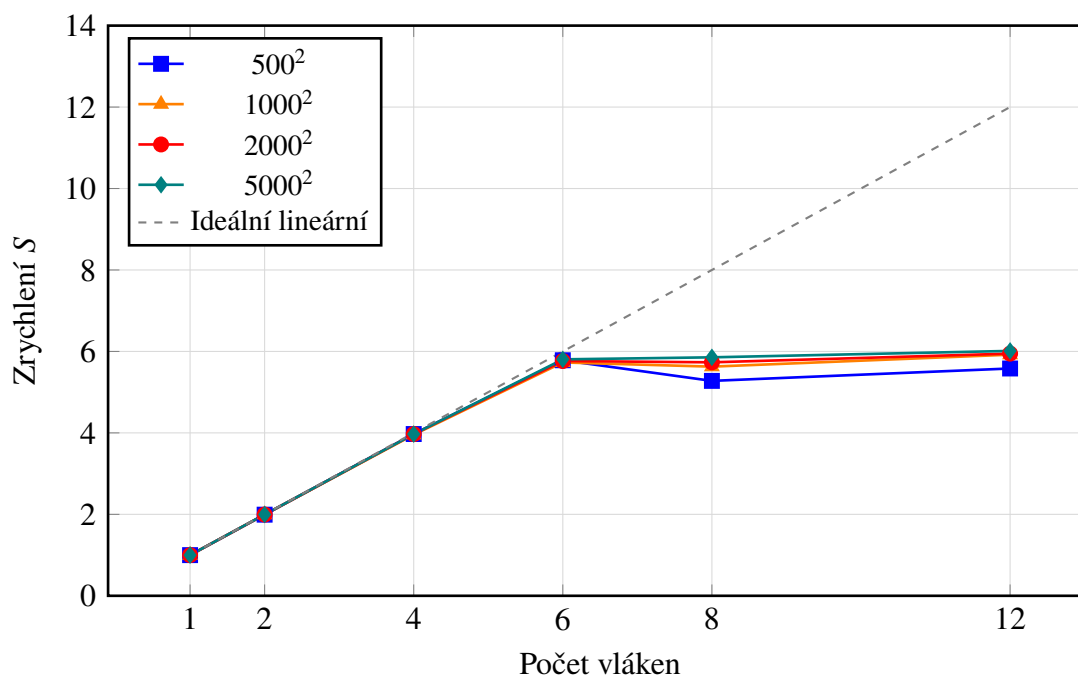
6.2.2 Škálování CPU paralelní varianty

Tabulka č. 6.8: Zrychlení CPU paralelní varianty – Game of Life

Vlákna	100 ²	500 ²	1000 ²	2000 ²	5000 ²	10 000 ²
1	1,00	1,00	1,00	1,00	1,00	1,00
2	2,00	1,98	1,99	1,99	2,00	2,00
4	3,71	3,87	3,96	3,97	3,97	3,99
6	4,38	5,79	5,74	5,77	5,81	5,84
8	4,02	5,21	5,63	5,74	5,86	5,90
12 (max)	4,45	5,59	5,93	5,95	6,02	6,04

Zdroj: vlastní zpracování

Obrázek č. 6.6: Zrychlení CPU paralelní varianty vs. počet vláken – Game of Life



Zdroj: vlastní zpracování

Na rozdíl od násobení matic je škálování Game of Life na CPU velmi konzistentní napříč velikostmi. Maximální zrychlení se stabilizuje kolem 6× na 12 vláknech, což odpovídá 6 fyzickým jádrům procesoru. Hyperthreading (12 logických jader) zde nepřináší téměř žádný další benefit, neboť úloha je primárně omezena výpočetním výkonem jader.

6.2.3 GPU zrychlení

Tabulka č. 6.9: Zrychlení GPU variant oproti sekvenční CPU – Game of Life

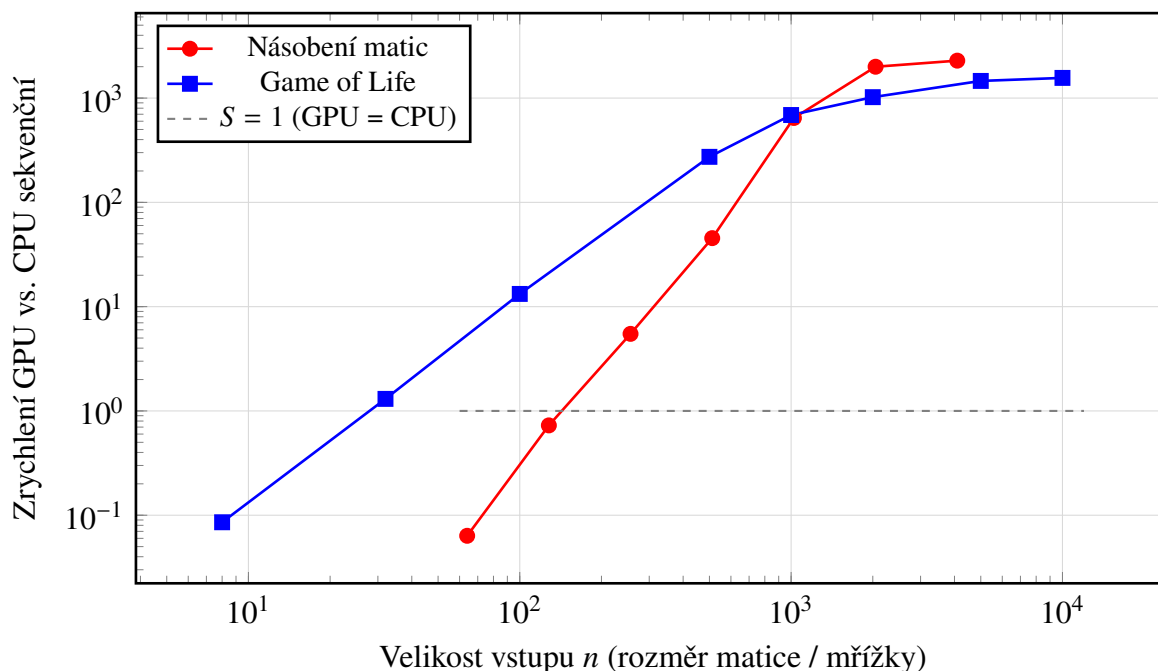
Rozměr mřížky	ILGPU	ComputeSharp
8^2	0,09	0,04
32^2	1,39	0,69
100^2	13,71	7,07
500^2	273,39	85,35
1000^2	688,13	113,78
2000^2	1020,70	601,46
5000^2	1460,57	863,11
$10\,000^2$	1561,35	729,33

Zdroj: vlastní zpracování

GPU zrychlení u Game of Life kontinuálně roste s velikostí mřížky, od $0,09\times$ (GPU pomalejší) pro nejmenší mřížku 8^2 po $1561\times$ pro $10\,000^2$. Na rozdíl od násobení matic zde nedochází k poklesu zrychlení pro velké vstupy, protože Game of Life je čistě výpočetně omezená úloha s lineární závislostí přístupu do paměti.

6.3 Souhrnné srovnání CPU a GPU

Obrázek č. 6.7: GPU zrychlení (ILGPU) obou úloh v závislosti na velikosti vstupu



Zdroj: vlastní zpracování

Z obrázek 6.7 vyplývají dva klíčové poznatky:

1. **Crossover point** závisí na poměru výpočetní složitosti k objemu přenášených dat. U násobení matic (crossover kolem $n = 140$) je pro malé matice výpočet příliš krátký oproti GPU overheadu. U Game of Life leží crossover kolem mřížky 28^2 ; pro větší mřížky 100 iterací zajistí dostatečný poměr náročnosti výpočtu ku velikosti přenášených dat.
2. **Růst zrychlení** je u násobení matic strmější (dosahuje maxima $6534\times$ v implementaci ComputeSharp a $2285\times$ v implementaci ILGPU pro 4096^2 matice), protože výpočetní složitost roste kubicky ($O(n^3)$), zatímco objem přenášených dat pouze kvadraticky, takže poměr užitečného výpočtu k režii přenosu se s rostoucím n zlepšuje. U Game of Life je výpočet $100 \cdot O(n^2)$ a data rovněž $O(n^2)$, tedy poměr výpočtu k přenosu zůstává konstantní (přibližně 100). Zrychlení proto roste pozvolněji (maximum $1561\times$ pro $10\,000^2$), ale bez poklesu.

Na základě naměřených dat lze pro testovanou sestavu (Ryzen 5 3600 + RTX 3070) a knihovnu ILGPU (CUDA backend) formulovat následující pozorování: **GPU se vyplatí, pokud**

výpočetní čas na CPU překročí přibližně 3 ms, což odpovídá naměřenému GPU overheadu (inicializace kontextu, alokace bufferů, kopírování dat, spuštění kernelu) na daném hardware. Konkrétní hodnota overheadu závisí na GPU, použitém backendu a objemu přenášených dat. Pro kratší výpočty dominuje režie přenosu dat a inicializace GPU.

6.4 Porovnání s Amdahlovým zákonem

Na základě naměřeného zrychlení CPU paralelní varianty lze odhadnout podíl paralelizovatelné části programu P . Parametr P je odhadnut z měření na **6 vlákních** (všechna fyzická jádra procesoru), protože to představuje maximální reálný paralelismus bez hyperthreadingu. Na základě tohoto P se pak vypočítávají teoretické předpovědi Amdahlova zákona pro všechny testované počty vláken (1, 2, 4, 6, 8, 12) a porovnávají se se skutečně naměřeným zrychlením. Testy pro 8 a 12 vláken jsou z experimentálního hlediska přidány nad rámec fyzických jader, aby se posoudilo, jak efektivně hyperthreading přispívá k dalšímu zrychlení.

Referenčním během pro násobení matic je Parallel(1) (viz sekce 6.1), pro Game of Life sekvenční varianta.

Z Amdahlova zákona (rovnice (1.1)) a naměřeného zrychlení S pro N vláken platí:

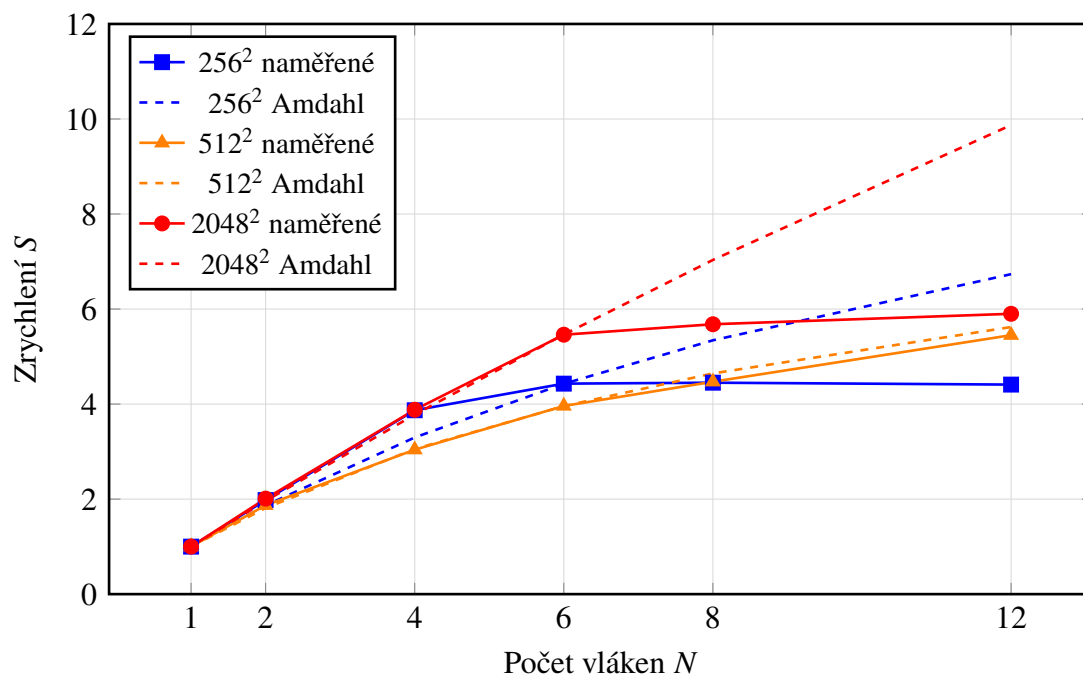
$$P = \frac{\frac{1}{S} - 1}{\frac{1}{N} - 1}$$

Tabulka č. 6.10: Odhad P z měření na 6 vlákních a porovnání predikce Amdahlova zákona pro 4 jádra (Parallel(1))

Úloha	Velikost	P (odhad)	Teor. $S(4)$	Skut. $S(4)$	Odchylka [%]
MatMul	256 ²	0,9289	3,30	3,87	17,3
MatMul	512 ²	0,8968	3,05	3,04	-0,3
MatMul	1024 ²	1,0000	4,00	4,46	11,5
MatMul	2048 ²	0,9804	3,78	3,88	2,6
MatMul	4096 ²	0,9324	3,32	2,60	-21,7
GoL	500 ²	0,9925	3,95	3,87	1,9
GoL	1000 ²	0,9906	3,94	3,96	0,5
GoL	2000 ²	0,9918	3,95	3,97	0,5
GoL	5000 ²	0,9933	3,96	3,97	0,3
GoL	10 000 ²	0,9943	3,97	3,99	0,5

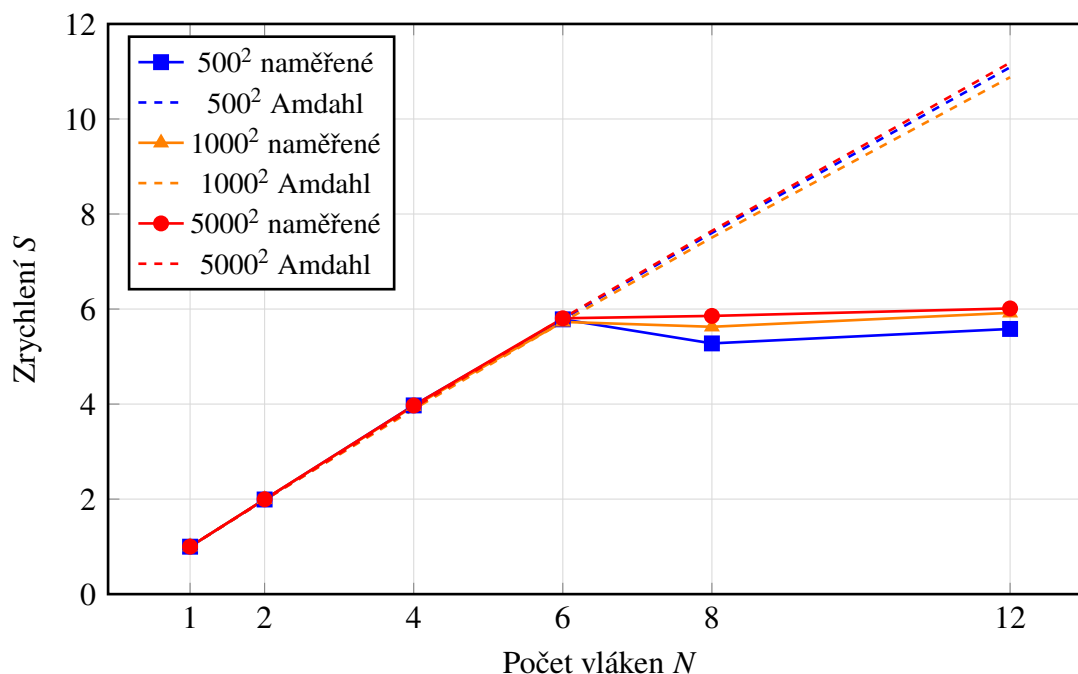
Zdroj: vlastní zpracování

Obrázek č. 6.8: Porovnání naměřeného zrychlení s predikcí Amdahlova zákona – násobení matic



Zdroj: vlastní zpracování

Obrázek č. 6.9: Porovnání naměřeného zrychlení s predikcí Amdahlova zákona – Game of Life



Zdroj: vlastní zpracování

Z výsledků Amdahlovy analýzy vyplývá:

- U Game of Life je odhad $P \approx 0,99$ stabilní napříč velikostmi (od 500^2 výše), úloha je konzistentně paralelizovatelná z přibližně 99 %. Predikce Amdahlova zákona pro 4 fyzická jádra odpovídá naměřeným hodnotám s odchylkou 1–2 %, což potvrzuje vysokou přesnost modelu v oblasti fyzických jader. Z grafů (obrázek 6.9) je zároveň patrné, že pro 8 a 12 vláken (s hyperthreadingem) teoretická predikce výrazně převyšuje naměřené hodnoty, protože hyperthreading nepřináší plný výkon dalšího jádra.
- U násobení matic se s referencí Parallel(1) odhad P pohybuje v rozsahu 0,90–0,98 pro většinu velikostí; výjimkou je 1024^2 , kde přetrvávající super-lineární zrychlení na vláknech 2–8 vede k $P = 1,0$. Predikce pro 4 jádra odpovídá naměřeným hodnotám s odchylkou do 3 % u matic 512^2 a 2048^2 ; matice 256^2 a 4096^2 vykazují vyšší odchylku vlivem nestability škálování, zejména 4096^2 s $-21,7\%$ odchylkou pravděpodobně kvůli omezení propustnosti paměti při velkém objemu dat. Příčina nadlineárního zrychlení 1024^2 zůstává bez profilování neznámá (viz sekce 7.4).

6.5 Porovnání s Gustafsonovým zákonem

Zatímco Amdahlův zákon předpokládá konstantní velikost úlohy a zkoumá, jak se mění čas výpočtu s přibývajícími procesory (silné škálování), Gustafsonův zákon předpokládá konstantní čas výpočtu a zkoumá, jak velkou úlohu zvládneme s více procesory. Tento přístup se nazývá *slabé škálování* (weak-scaling) a lépe odpovídá reálnému využití paralelních systémů, kde rostoucí výpočetní kapacita umožňuje řešit větší problémy.

Pro ověření Gustafsonova zákona byl proveden weak-scaling experiment, ve kterém se velikost úlohy škáluje s počtem vláken podle vzorců uvedených v sekci (4.3.5). Pro každou kombinaci počtu vláken a škálované velikosti bylo provedeno 10 běhů a vypočítány statistiky včetně mediánu paralelního času. Na základní konfiguraci (512^2) byl naměřen sekvenční čas 187,4 ms pro násobení matic a 1539,7 ms pro Game of Life, dále byla sekvenční varianta spuštěna na každé škálované velikosti, což umožnilo vypočítat zrychlení a sekvenční frakci α přímo z naměřených hodnot.

6.5.1 Násobení matic

Tabulka č. 6.11: Weak-scaling pro násobení matic ($n = 512 \cdot P^{1/3}$)

P	Velikost	Par. čas [ms]	Seq. čas [ms]	S_G	α	CV [%]
2	645^2	145,7	230,7	1,58	0,42	3,29
4	813^2	177,3	522,9	2,95	0,35	2,11
6	930^2	229,6	741,1	3,23	0,55	8,18
8	1024^2	463,6	5038,9	10,87	-0,41	3,14
12	1172^2	593,3	1543,1	2,60	0,85	4,59

Zdroj: vlastní zpracování

Použitý procesor má 6 fyzických jader. Konfigurace nad 6 vláken proto již neodpovídají přidávání dalších fyzických jader, ale využití logických vláken pomocí technologie Hyperthreading. Z tohoto důvodu jsou výsledky pro 8 a 12 vláken interpretovány opatrně, protože nelze očekávat stejné škálování jako při přidávání fyzických jader.

U násobení matic jsou výsledky experimentu méně stabilní než u Game of Life. Zrychlení roste do konfigurace $P = 6$ (dosahuje hodnoty 3,23 z ideálních 6) a výsledky jsou stabilní. Pro $P = 12$ se zrychlení opět snižuje vlivem překročení počtu fyzických jader procesoru, ale konfigurace $P = 8$ vykazuje abnormálně velkou hodnotu $S_G = 10,87$, která překračuje ideální zrychlení $S_I = 8$. Toto je způsobeno anomálií v měření sekvenčního výpočtu pro matici 1024^2 , kde naměřený čas běhu je

abnormálně dlouhý. Považujeme toto měření za chybné. Pokud pro demonstraci použijeme základní sekvenční čas $t_{seq,base} = 203,3981$ ms a přepočít pro $P = 8$,

$$t_{seq,8}^* = t_{seq,base} \cdot 8 = 203,3981 \cdot 8 = 1627,1848 \text{ ms},$$

pak vychází

$$S_G^*(8) = \frac{t_{seq,8}^*}{t_{par,8}} = \frac{1627,1848}{463,6} = 3,51,$$

a odpovídající sekvenční frakce

$$\alpha^* = \frac{P - S_G^*}{P - 1} = \frac{8 - 3,51}{7} = 0,64.$$

Tato hodnota je realistická a potvrzuje, že skok při $P = 8$ v původní tabulce je důsledek nestabilního sekvenčního měření, nikoliv reálného překonání lineárního škálování.

6.5.2 Game of Life

Tabulka č. 6.12: Weak-scaling pro Game of Life ($n = 512 \cdot \sqrt{P}$)

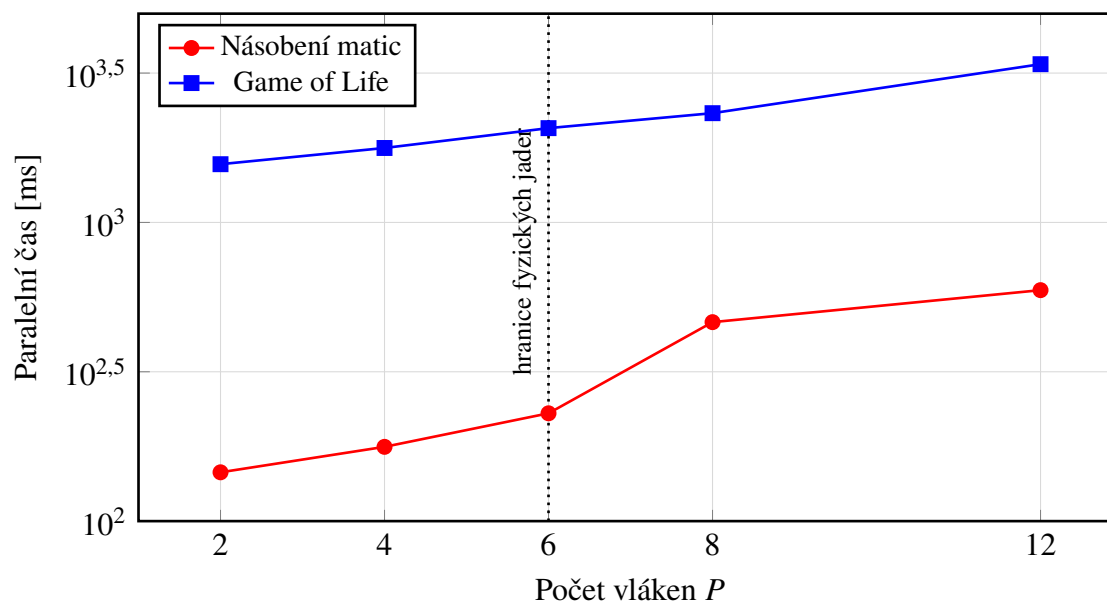
P	Velikost	Par. čas [ms]	Seq. čas [ms]	S_G	α	CV [%]
2	724 ²	1566,7	3088,5	1,97	0,03	1,29
4	1024 ²	1775,2	6153,3	3,47	0,18	2,99
6	1254 ²	2068,6	9234,7	4,46	0,31	1,65
8	1448 ²	2321,5	12 130,1	5,23	0,40	1,24
12	1774 ²	3386,7	18 162,7	5,36	0,60	5,32

Zdroj: vlastní zpracování

Game of Life poskytuje konzistentnější výsledky. Zrychlení S_G roste monotónně z hodnoty přibližně 1,97 při $P = 2$ na 5,36 při $P = 12$. Růst je však sublineární, což ukazuje, že paralelní implementace sice dokáže využít rostoucí počet vláken, ale efektivita s vyšším P postupně klesá. Tomu odpovídá růst odhadované hodnoty α_{strict} , která se zvyšuje z pouhých 0,03 při $P = 2$ na 0,60 při $P = 12$.

6.5.3 Grafické porovnání

Obrázek č. 6.10: Weak-scaling čas výpočtu

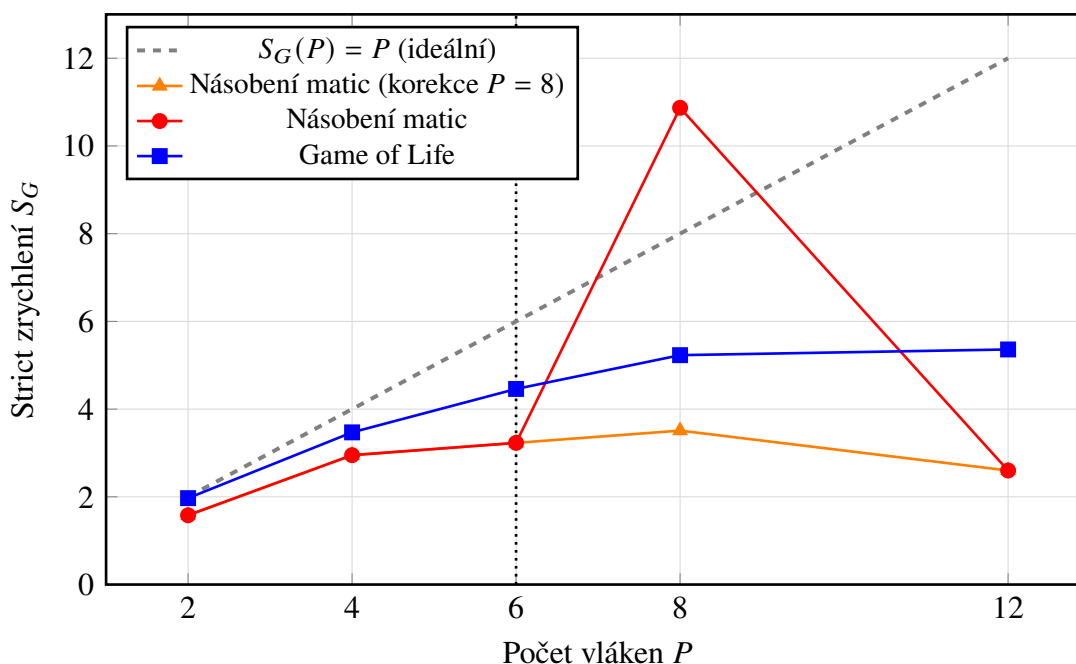


Zdroj: vlastní zpracování

Graf 6.10 zobrazuje vývoj paralelního času výpočtu v logaritmickém měřítku. Podle Gustafsonova zákona by ideální škálování znamenalo konstantní čas výpočtu při rostoucí velikosti úlohy, každé zvýšení P je kompenzováno odpovídajícím zvětšením problému. Naměřené výsledky však ukazují monotónní růst času u obou algoritmů.

U Game of Life (modrá křivka) čas roste z 1566,7 ms ($P = 2$) na 3386,7 ms ($P = 12$), tedy 2,16×. Tento relativně mírný nárůst indikuje, že se algoritmus dobře škáluje, při zdvojnásobení pracovní zátěže se čas nezvyšuje proporcionálně. Násobení matic (červená křivka) vykazuje výraznější variabilitu: čas roste z 145,7 ms ($P = 2$) na 593,3 ms ($P = 12$), což představuje 4,07× nárůst.

Obrázek č. 6.11: Zrychlení vůči ideálnímu Gustafsonovu škálování



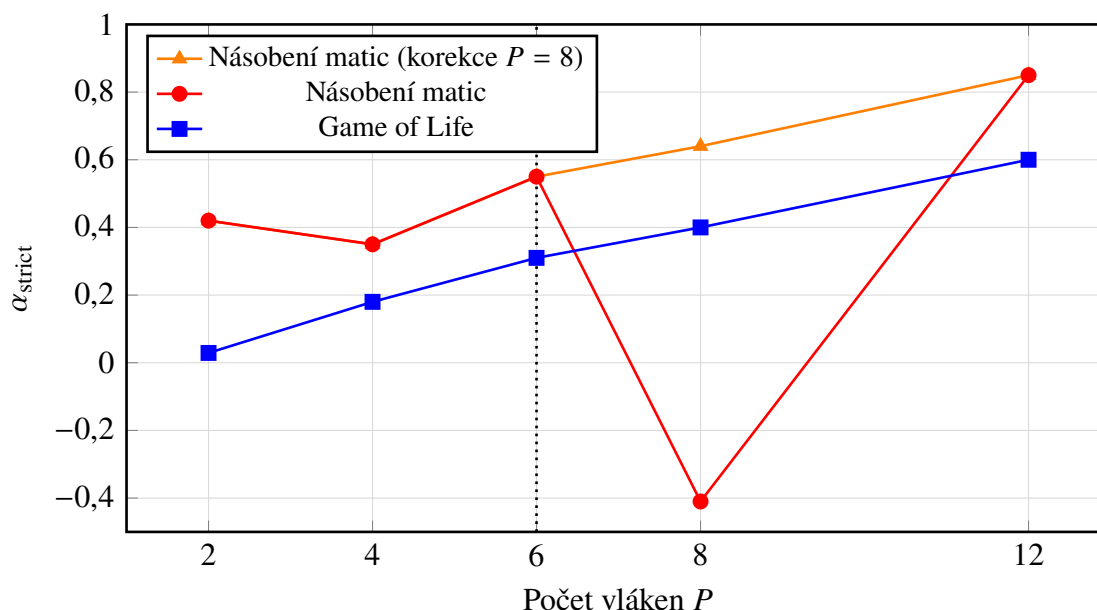
Zdroj: vlastní zpracování

Graf 6.11 je klíčový pro kvantitativní posouzení Gustafsonova škálování. Šedá čárkovaná přímka $S_G(P) = P$ reprezentuje ideální Gustafsonův zákon, kde se zrychlení rovná počtu procesorů. Reálné implementace z principu dosahují nižších hodnot kvůli komunikační režii, synchronizaci a sekvenčním částem algoritmu.

Game of Life (modrá křivka) vykazuje nejlepší aproximaci ideálu. V rozsahu fyzických jader ($P = 2$ až $P = 6$) dosahuje 74,3 % ideálního zrychlení v průměru ($1,97/2 = 98,5\%$ pro $P = 2$, $4,46/6 = 74,3\%$ pro $P = 6$). Po překročení hranice fyzických jader efektivita klesá. Při $P = 12$ dosahuje $S_G = 5,36$, tedy pouze 44,7 % ideálu.

Násobení matic (červená křivka) vykazuje výrazně nestabilnější průběh. Do $P = 6$ se hodnoty drží v rozmezí $S_G \in [1,58; 3,23]$ (26–54 % ideálu), což je výrazně horší než u Game of Life. Oranžová křivka znázorňuje korektní hodnotu $S_G^* = 3,51$ pro $P = 8$ po odstranění anomálie, tato hodnota je konzistentní s trendem pozorovaným u ostatních konfigurací. Pokles na $S_G = 2,60$ při $P = 12$ naznačuje, že násobení matic má vyšší sekvenční režii a hůře toleruje přechod na logická vlákna.

Obrázek č. 6.12: Odhad sekvenční frakce α podle Gustafsonova zákona



Zdroj: vlastní zpracování

Graf 6.12 znázorňuje vývoj sekvenční frakce α vypočtené podle vztahu $\alpha = (P - S_G) / (P - 1)$. Tato metrika poskytuje dobrý pohled na efektivitu škálování, nižší hodnota α znamená menší sekvenční režii a lepší paralelizovatelnost.

U Game of Life (modrá křivka) hodnota α monotónně roste z 0,03 ($P = 2$) na 0,60 ($P = 12$). Extrémně nízká hodnota 0,03 při $P = 2$ indikuje téměř perfektní paralelizovatelnost na malém počtu vláken, pouze 3 % sekvenční režii. S rostoucím P však overhead narůstá, což se projevuje růstem α .

Násobení matic (červená + oranžová křivka) vykazuje vyšší a nestabilnější hodnoty α . V rozsahu $P \in \{2, 4, 6\}$ kolísá kolem 0,42–0,55 (oproti 0,03–0,31 u GoL), což indikuje výrazně větší sekvenční režii. Oranžová křivka ukazuje korektní hodnotu $\alpha^* = 0,64$ pro $P = 8$ po odstranění anomálie.

6.5.4 Shrnutí

Výsledky Gustafsonovy analýzy (weak-scaling) ukazují rozdílné chování obou algoritmů. Game of Life vykazuje stabilnější a lépe interpretovatelné škálování s zrychlením $S_G \in [1,97; 5,36]$ a monotónním růstem sekvenční frakce $\alpha \in [0,03; 0,60]$. Algoritmus efektivně využívá až 6 fyzických jader (74 % ideálního Gustafsonova zrychlení), ale efektivita klesá při využití multithreadingu na 45 % při $P = 12$.

Násobení matic vykazuje výraznější nestabilitu s $S_G \in [1,58; 3,23]$ (bez anomálie $P = 8$) a vyšší sekvenční frakcí $\alpha \in [0,35; 0,85]$. Algoritmus dosahuje pouze 26–54 % ideálního zrychlení na fyzických jádrech a degraduje na 22 % při $P = 12$. Anomálie při $P = 8$ (naměřené $S_G = 10,87$) byla identifikována jako nestabilní sekvenční měření; korektní hodnota $S_G^* = 3,51$ odpovídá očekávanému trendu.

6.6 Porovnání ILGPU a ComputeSharp

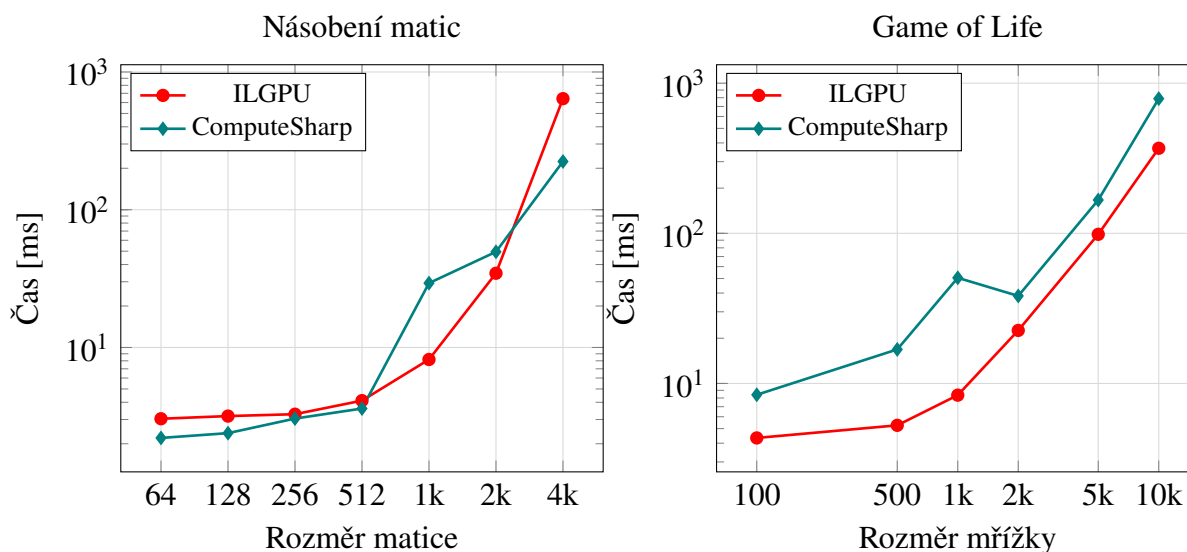
Obě GPU knihovny byly testovány na stejném hardware a se stejnými vstupními daty. Tabulka 6.13 uvádí porovnání jejich výkonu.

Tabulka č. 6.13: Porovnání ILGPU a ComputeSharp – časy výpočtu [ms] a poměr

Úloha	Velikost	ILGPU [ms]	ComputeSharp [ms]	Poměr CS/ILGPU
MatMul	64 ²	3,04	2,20	0,72
MatMul	128 ²	3,20	2,39	0,75
MatMul	256 ²	3,28	3,05	0,93
MatMul	512 ²	4,11	3,60	0,88
MatMul	1024 ²	8,18	29,32	3,58
MatMul	2048 ²	34,59	49,49	1,43
MatMul	4096 ²	640,33	223,96	0,35
GoL	100 ²	4,34	8,40	1,94
GoL	500 ²	5,26	16,85	3,20
GoL	1000 ²	8,36	50,54	6,05
GoL	2000 ²	22,55	38,27	1,70
GoL	5000 ²	98,47	166,63	1,69
GoL	10 000 ²	368,47	788,81	2,14

Zdroj: vlastní zpracování

Obrázek č. 6.13: Porovnání ILGPU a ComputeSharp



Zdroj: vlastní zpracování

Z porovnání obou knihoven vyplývá:

- **Násobení matic (malé velikosti):** ComputeSharp je pro matice do velikosti 512^2 srovnatelný s ILGPU (poměr 0,72–0,93×), přičemž pro nejmenší matice je mírně rychlejší díky nižšímu overhead DirectX 12 backendu.
- **Násobení matic (1024^2):** ILGPU je výrazně rychlejší (8,18 vs 29,32 ms), což naznačuje, že CUDA PTX kód je efektivnější pro středně velké výpočty.
- **Násobení matic (4096^2):** ComputeSharp je naopak 2,9× rychlejší než ILGPU (223,96 vs 640,33 ms), což může souviset s efektivnější správou paměti v DirectX 12 pro velké datové sady.
- **Game of Life:** ILGPU je konzistentně rychlejší (1,7–6×), přičemž rozdíl roste s počtem iterací. Příčinou je pravděpodobně efektivnější práce s ping-pong double bufferingem v CUDA a nižší overhead synchronizace mezi 100 iteracemi.

7 Diskuse

Výsledky z předchozí kapitoly je potřeba zasadit do širšího kontextu, porovnat s teoretickými předpoklady, identifikovat faktory ovlivňující reálný výkon a zhodnotit zvolenou technologii.

7.1 Hlavní zjištění

Měření potvrdila, že GPU implementace algoritmů v knihovnách ILGPU a ComputeSharp přinášejí řádové zrychlení pro dostatečně velké vstupy. Nejvyšší naměřené zrychlení GPU oproti sekvenční CPU bylo $6534\times$ pro násobení matic 4096^2 v implementaci ComputeSharp. Varianta ILGPU dosáhla pro stejnou velikost zrychlení $2285\times$. U Game of Life bylo dosaženo zrychlení $1561\times$ pro mřížku $10\,000^2$. I oproti CPU paralelní variantě s 12 vlákny představuje GPU zrychlení $222\times$ (MatMul 2048^2) a $259\times$ (GoL $10\,000^2$).

CPU paralelní varianta dosáhla maximálního zrychlení $14,7\times$ na 12 vláknech (MatMul 1024^2 , super-lineární v důsledku anomálně pomalého sekvenčního běhu; viz sekce 7.4) a stabilních $6\times$ u Game of Life (odpovídá 6 fyzickým jádrům procesoru).

7.2 CPU versus GPU – crossover point

Crossover point se výrazně liší mezi oběma úlohami:

- **Násobení matic:** GPU (ILGPU) se stává rychlejší než sekvenční CPU přibližně pro matice 140×140 a rychlejší než paralelní CPU (12 vláken) kolem velikosti 256^2 . Pro nejmenší testovanou matici 64^2 je GPU $16\times$ pomalejší (3,04 ms vs 0,19 ms).
- **Game of Life:** GPU je rychlejší než sekvenční CPU **pro všechny testované velikosti od 32^2** , včetně mřížky 32^2 (4,42 ms vs 6,16 ms sekvenčně). Pro menší mřížky (8^2 , 24^2) je GPU pomalejší kvůli dominantnímu overheadu.

Rozdíl je způsoben poměrem výpočetní složitosti k objemu přenášených dat. U násobení matic $n \times n$ je výpočetní složitost $O(n^3)$ a objem dat $O(n^2)$, takže poměr roste s n . Pro malé n výpočet nestačí pokrýt GPU overhead (přibližně 3 ms). U Game of Life 100 iterací zajistí poměr výpočtu k přenosu $O(100 \cdot n^2)/O(n^2) = 100$, což je dostatečné i pro nejmenší mřížku.

Na základě naměřených dat lze pro testovanou sestavu formulovat pozorování, že GPU se vyplatí přibližně od okamžiku, kdy výpočetní čas na CPU přesáhne GPU overhead (≈ 3 ms). Samotný overhead je dán především hardwarem, použitým backendem a objemem přenášených dat; na jiném GPU či s jinou knihovnou se prahová hodnota může lišit.

7.3 Vliv overheadu přenosu dat CPU–GPU

GPU overhead je tvořen několika složkami: (1) alokace paměťových bufferů na GPU, (2) kopírování vstupních dat z CPU na GPU, (3) spuštění kernelu (launch latency), (4) kopírování výsledků zpět na CPU. V našich měřeních byl celkový overhead přibližně 3 ms, ILGPU pro MatMul 64^2 naměřil 3,04 ms, přičemž samotný výpočet na CPU trval 0,19 ms.

S rostoucí velikostí vstupu se overhead stává zanedbatelným. Pro MatMul 2048^2 tvoří overhead (odhadem stále ≈ 3 ms) pouze 9 % z celkového GPU času 34,59 ms, zatímco výpočet tvoří 91 %. U Game of Life je situace ještě příznivější, protože data se přenáší jednou, ale výpočet probíhá ve 100 iteracích.

Zajímavým pozorováním je, že ComputeSharp vykazuje pro malé matice (do 512^2) **nižší** overhead než ILGPU (2,20 ms vs 3,04 ms pro 64^2), ale pro Game of Life je naopak pomalejší (8,40 ms vs 4,34 ms pro 100^2). To naznačuje, že DirectX 12 backend má nižší latenci spuštění jediného kernelu, ale vyšší overhead při opakovaném volání kernelu v iterační smyčce.

7.4 Anomálie v datech

7.4.1 Super-lineární zrychlení MatMul 1024^2

Násobení matic 1024^2 vykazuje super-lineární zrychlení $14,7\times$ na 12 vláknech oproti sekvenční variantě (Amdahlova analýza dává $P = 1,0$). Super-lineární zrychlení zde nevzniká tím, že by paralelní varianta byla mimořádně efektivní, ale tím, že **sekvenční varianta je při této konkrétní velikosti abnormálně pomalá**. Část efektu je způsobena rozdílem v JIT kompilaci – Parallel(1) je $1,3\times$ rychlejší než sekvenční varianta. I po normalizaci na Parallel(1) však zůstává zrychlení super-lineární na vláknech 2–8 ($2,23$ – $8,29\times$); teprve při 12 vláknech ($11,28\times$) je sub-lineární (hyper-threading).

Bez profilování na úrovni hardware performance counters nelze příčinu anomálie určit s jistotou. Skutečnost, že Parallel(1) na jednom jádře je $1,3\times$ rychlejší než sekvenční varianta, naznačuje, že zpomalení sekvenčního běhu nesouvisí primárně s počtem jader, ale pravděpodobně s rozdíly v generovaném JIT kódu či přístupových vzorech paralelního frameworku. I po normalizaci na Parallel(1) však přetrvává super-lineární zrychlení na vláknech 2–8, což ukazuje na další faktor související s víceprocesorovým zpracováním. Přesná povaha tohoto efektu, ať už jde o chování paměťové hierarchie, plánování úloh nebo jiný mechanismus, by vyžadovala detailní profilování, které je nad rámec této práce.

7.4.2 Propad zrychlení MatMul 4096²

Na opačném konci spektra stojí matice 4096², kde celkový objem dat (tři matice $\times 4096^2 \times 4 \text{ B} \approx 192 \text{ MB}$) výrazně přesahuje L3 cache (32 MB), což naznačuje, že výpočet se při této velikosti stává převážně *memory-bound*.

Oproti sekvenční variantě dosahuje CPU paralelní varianta na 6 vláknech zrychlení 10,3 $\times$, avšak na 8 vláknech (s hyperthreadingem) klesá na 7,9 $\times$. Tato vysoká čísla jsou však zkreslena abnormálně pomalým sekvenčním během – Parallel(1) je u této velikosti 2,3 $\times$ rychlejší než sekvenční varianta. Po normalizaci na Parallel(1) činí zrychlení na 6 vláknech 4,5 $\times$, na 8 vláknech klesá na 3,5 $\times$ a na 12 vláknech se vrací na 4,4 $\times$ (tabulka 6.4). Škálování je tedy výrazně sub-lineární a propad na 8 vláknech naznačuje, že hyperthreading u takto paměťově náročných úloh výkon nepřináší, pravděpodobně kvůli souperení o šířku pásma paměťového subsystému.

GPU zrychlení (ILGPU) pro 4096² dosahuje 2285 $\times$ oproti sekvenční CPU, tedy více než pro 2048² (1998 $\times$), protože GPU disponuje vlastní vysokorychlostní pamětí nezávislou na CPU cache hierarchii. Sekvenční čas pro tuto velikost vykazoval vysokou variabilitu ($CV = 8,38 \%$), což může souviset se stránkováním paměti či plánováním operačního systému.

7.4.3 Anomálie ve weak-scaling experimentu ($P = 8$, násobení matic)

Ve weak-scaling experimentu pro násobení matic vykazuje konfigurace $P = 8$ zjevně nereálný Gustafsonův speedup $S_G = 10,87$ a zápornou sekvenční frakci $\alpha = -0,41$ (viz tabulka 6.11). Příčinou je anomálně dlouhý sekvenční běh pro škálovanou velikost $n \approx 1024$, tedy stejná anomálie sekvenční varianty, která je popsána výše (super-lineární zrychlení 1024²). Weak-scaling vzorec pro $P = 8$ dává $n = 512 \cdot 8^{1/3} \approx 1024$, čímž se sekvenční referenční měření dostává přesně do oblasti postižené touto anomálií. Alternativní přepočít s opravou sekvenčního času (viz kapitola 6) dává realistický odhad $S_G^* \approx 3,51$ a $\alpha^* \approx 0,64$, v souladu s trendem ostatních konfigurací. Kompletní statistiky včetně směrodatných odchylek jsou uvedeny v příloze (tabulky B.13 and B.14).

7.4.4 Konzistence Game of Life

Game of Life naproti tomu vykazuje velmi konzistentní škálování: $P \approx 0,99$ (Amdahl, z 6 vláken) a $S_{max} \approx 5,8$ na 6 vláknech nezávisle na velikosti mřížky od 500² výše. Weak-scaling experiment potvrdil konzistentní slabé škálování, s Gustafsonovým speedupem 1,97 až 5,36 (2 až 12 vláken) a sublineárním růstem paralelního času. To potvrzuje, že Game of Life je čistě *compute-bound* úloha s minimální variabilitou v přístupových vzorech a stabilní sekvenční frakcí.

7.5 Interpretace Amdahlova zákona

Amdahlův zákon se ukázal jako dobrý prediktor pro Game of Life v rozsahu fyzických jader (do 6 vláken), kde odchylka mezi predikcí a naměřenou hodnotou byla prakticky nulová. Stabilní $P \approx 0,99$ (z 6 vláken) naznačuje, že přibližně 1 % výpočetního času tvoří sekvenční overhead (inicializace polí, správa vláken, synchronizace na konci každé iterace). Pro 8 a 12 vláken (zahrnujících hyperthreading) však teoretická predikce výrazně převyšuje naměřené hodnoty (odchylka 23–31 %), což potvrzuje, že Amdahlův zákon předpokládá ideální procesory a hyperthreading nepřináší plný výkon dalšího jádra.

U násobení matic je situace složitější. S referencí Parallel(1) se odhad P pohybuje v rozsahu 0,90–0,98 pro většinu velikostí; pouze matice 1024^2 dosahuje $P = 1,0$ v důsledku anomálie popsané v sekce 7.4. Při použití sekvenční varianty jako reference vychází $P = 1,0$ i pro 2048^2 a 4096^2 , což je však zkreslení způsobené pomalejším sekvenčním během. Anomálie sekvenčního běhu mění efektivní poměr sekvenční a paralelní fáze, a proto je Amdahlův zákon vhodný pro úlohy se stabilním a predikovatelným škálováním, ale méně spolehlivý tam, kde sekvenční reference vykazuje nestabilní chování.

7.6 Interpretace Gustafsonova zákona

Gustafsonův zákon předpokládá, že s rostoucím počtem procesorů se škáluje velikost úlohy tak, aby výpočetní čas zůstal přibližně konstantní. Teorie uvádí vztah $S_G(N) = N - \alpha(N - 1)$, kde α je sekvenční frakce běhu. Pro ověření tohoto modelu byl proveden weak-scaling experiment se škálovanými velikostmi vstupů podle počtu vláken (viz sekce 4.3.5). Procesor má 6 fyzických jader; konfigurace s 8 a 12 vlákny využívají logická vlákna prostřednictvím technologie Hyper-Threading.

Game of Life vykazuje konzistentnější weak-scaling chování. Gustafsonův speedup roste z 1,97 (2 vlákna) na 5,36 (12 vláken) při škálované velikosti 1774^2 , což je 44,7 % teoretického ideálu $S_G(12) = 12$. Paralelní čas se zvyšuje pozvolna z 1566,7 ms (2 vlákna, 724^2) na 3386,7 ms (12 vláken, 1774^2), což celkově reprezentuje nárůst 2,16krát - příznivě menší než lineární nárůst dat (2,45krát z 724^2 na 1774^2). Hodnoty α_{strict} rostou od 0,029 (2 vlákna) po 0,60 (12 vláken), což ukazuje klesající efektivitu s vyšším využitím hyperthreadingu. Zlom lze pozorovat kolem $P = 6$ odpovídajícímu fyzickým jádrům. Game of Life tak potvrzuje předpoklad slabého škálování: úloha se zvětší úměrně s počtem procesorů, přičemž čas výpočtu roste sublineárně. Toto chování je typické pro compute-bound úlohy bez paměťových anomálií.

Násobení matic naproti tomu vykazuje méně stabilní weak-scaling chování. Paralelní čas roste neproporcionálně: Od 145,7 ms (2 vlákna, 645^2) na 463,6 ms (8 vláken, 1024^2), což je skok 3,2krát pro zvětšení dat pouze 2,5krát. Na 12 vláknech se čas částečně zotavuje na 593,3 ms. Pokles efektivnosti je patrný z α_{strict} , která kolísá mezi 0,35 a 0,85 s výjimkou $P = 8$, kde dosahuje negativní

hodnoty $\alpha_{\text{strict}} = -0,41$. Tato záporná hodnota není interpretována jako reálná záporná sekvenční frakce, nýbrž jako důsledek anomálně pomalého sekvenčního běhu pro velikost okolo 1024^2 , jehož přesná příčina nebyla bez profilování určena. Podíl CV na 6 vláknech dosahuje 8,18 %, což naznačuje měřitelnou nestabilitu. Gustafsonův předpoklad konstantní sekvenční frakce se u násobení matic neplní dostatečně konzistentně. Weak-scaling experiment pro násobení matic je proto užitečný především jako orientační srovnání, nikoliv jako přesné ověření teoretického modelu.

7.7 Srovnání ILGPU a ComputeSharp

Porovnání obou GPU knihoven přineslo několik poznatků:

1. **Kernel launch overhead:** ComputeSharp (DirectX 12) má nižší overhead pro jednorázové spuštění kernelu, u malých matic (do 512^2) je srovnatelný nebo mírně rychlejší než ILGPU (CUDA).
2. **Iterační výkon:** ILGPU je konzistentně 1,7–6× rychlejší u Game of Life, kde se kernel spouští 100× v cyklu. CUDA PTX backend je efektivnější pro opakované spuštění kernelů se synchronizací.
3. **Velké matice** (4096^2): ComputeSharp je pro největší matice 2,9× rychlejší než ILGPU (223,96 ms vs 640,33 ms), což může souviset s efektivnější správou paměti v DirectX 12 backendu pro velké datové sady.
4. **Velké vstupy:** Původní 1D dispatch ComputeSharp narážel na limit 65 535 thread groups pro velké vstupy. Po přechodu na 2D dispatch (viz sekce 5.5) jsou data dostupná pro všechny testované velikosti.
5. **Programovací model:** ComputeSharp využívá source generátory a atribut [Generated-ComputeShaderDescriptor], což zjednodušuje konfiguraci, ale omezuje flexibilitu oproti ILGPU, které nabízí přímou kontrolu nad alokací, kopírováním a synchronizací.

Souhrnně: ILGPU je vhodnější pro výpočetně náročné úlohy s iterativními výpočty (Game of Life). ComputeSharp se hodí pro jednorázové zpracování menších datasetů a překvapivě i pro největší matice (4096^2), zejména tam, kde je důležitá platformní nezávislost (DirectX 12 běží na libovolném Windows zařízení s GPU, bez nutnosti NVIDIA hardware).

7.8 Srovnání C# ekosystému s alternativami

Podrobné srovnání dostupných technologií pro GPU výpočty je uvedeno v kapitola 3. Na základě praktických zkušeností z této práce lze ke srovnání dodat několik poznatků.

ILGPU generuje PTX kód, který je pro jednoduché kernely výkonově srovnatelný s kódem z `nvcc`. Naměřená zrychlení GPU potvrzují, že režie .NET vrstvy je zanedbatelná oproti samotnému výpočtu. Při implementaci `ComputeSharp` se ukázalo praktické omezení DirectX 12 backendu v podobě limitu 65 535 thread groups na dimenzi, které vyžadovalo přechod z 1D na 2D dispatch (viz sekce 5.5). Tento typ problému u ILGPU s CUDA backendem nevzniká.

Pro pokročilé optimalizace (sdílená paměť, warp-level primitiva) zůstává nativní CUDA v C/C++ vhodnější volbou. C# s ILGPU však představuje dobrý kompromis mezi pohodlím vývoje a výkonem pro úlohy, kde naivní kernel stačí.

7.9 Omezení práce

Výsledky práce je třeba interpretovat s vědomím několika omezení:

- **Jediný hardware:** Měření probíhalo na jedné sestavě (Ryzen 5 3600 + RTX 3070). Na jiném hardware (jiný poměr CPU/GPU výkonu, jiná architektura GPU) by se crossover pointy a zrychlení mohly výrazně lišit.
- **Naivní algoritmus:** Všechny varianty implementace záměrně používají stejný naivní algoritmus se třemi vnořenými smyčkami (viz sekce 5.2), aby srovnání odráželo rozdíl v architektuře, nikoliv v algoritmu. Optimalizované knihovny (`cuBLAS`, `BLAS`) by dosáhly vyššího absolutního výkonu, ale znemožnily by izolaci vlivu platformy.
- **Jediný typ GPU:** Měření probíhalo výhradně na NVIDIA GPU s CUDA backendem. Výsledky pro OpenCL backend ILGPU na AMD GPU mohou být odlišné.
- **Dvě úlohy:** Testovány byly pouze dva typy úloh (compute-bound s kubickou a kvadratickou složitostí). Jiné vzory (scatter/gather, redukce, Monte Carlo) by mohly vykazovat odlišné charakteristiky.
- **Variabilita měření:** Zejména pro velké matice (4096^2) byla variabilita běhů zvýšená (u sekvencí varianty $CV = 8,38\%$), což ovlivňuje spolehlivost odhadů zrychlení a Amdahlovy analýzy pro tuto velikost.

7.10 Možná rozšíření a budoucí práce

V navazujícím výzkumu by bylo zajímavé:

- Implementovat optimalizované GPU kernely se sdílenou pamětí (tiled MatMul) a porovnat zrychlení oproti naivní verzi.
- Porovnat naivní implementaci s optimalizovanými knihovnami (cuBLAS pro MatMul) pro kvantifikaci rozdílu mezi vlastní a knihovnoví implementací.
- Testovat na GPU různých generací a výrobců (AMD s OpenCL backendem ILGPU) pro ověření přenositelnosti.
- Přidat další typy úloh: Monte Carlo simulaci (embarrassingly parallel), redukci (communication-heavy) a Mandelbrotovu množinu (load-imbalanced).
- Provést přímé srovnání se stejnými kernely napsanými v nativní CUDA C++ pomocí nvcc pro kvantifikaci overhead .NET/ILGPU vrstvy.
- Prozkoumat využití warp-level primitiv a kooperativních skupin dostupných v ILGPU od verze 1.5.

Závěr

Cílem této diplomové práce bylo prozkoumat možnosti paralelizace výpočtů, implementovat vybrané výpočetně náročné úlohy ve třech variantách a změřit jejich výkon. U CPU paralelní varianty byla měřena závislost zrychlení na počtu vláken (1 až 12) a analyzována pomocí Amdahlova a Gustafsonova zákona. U GPU implementací byly porovnány dvě knihovny (ILGPU a ComputeSharp) v jejich automatických konfiguracích a identifikovány crossover pointy mezi CPU a GPU.

V teoretické části práce byly popsány základní koncepty paralelních výpočtů, včetně Flynnovy taxonomie, Amdahlova a Gustafsonova zákona. Byla podrobně rozebrána architektura GPU, její paměťová hierarchie, execution model a rozdíly oproti CPU. Byl proveden přehled dostupných technologií pro GPU výpočty, od nízkoúrovňových platform CUDA a OpenCL přes nástroje v C#/.NET (ILGPU, ComputeSharp, ManagedCUDA, Silk.NET) až po Python knihovny (CuPy, Numba, PyCUDA).

V praktické části byly implementovány dvě úlohy, násobení matic a Game of Life, ve třech variantách: sekvenční CPU, paralelní CPU a GPU. Pro implementaci byl zvolen jazyk C# s knihovnou ILGPU, která umožňuje psát GPU kernely přímo v C# bez nutnosti znalosti CUDA C++.

Výsledky měření potvrdily, že GPU paralelizace přináší řádové zrychlení pro dostatečně velké vstupy. U násobení matic bylo dosaženo nejvyššího zrychlení $6534\times$ oproti sekvenční CPU variantě pro matice 4096^2 v implementaci ComputeSharp. Varianta ILGPU dosáhla pro stejnou velikost zrychlení $2285\times$. U Game of Life bylo dosaženo zrychlení $1561\times$ pro mřížku $10\,000^2$. I oproti CPU paralelní variantě s 12 vlákny přineslo GPU zrychlení $222\times$ (MatMul) a $259\times$ (GoL).

Byl identifikován crossover point pro obě úlohy: U násobení matic je GPU rychlejší od velikosti matice přibližně 140×140 . U Game of Life leží crossover point přibližně mezi mřížkami 24^2 a 32^2 ; pro větší velikosti je GPU vždy rychlejší.

CPU paralelní varianta dosáhla maximálního zrychlení $6\times$ na 12 vláknech u Game of Life (odpovídá 6 fyzickým jádrům procesoru) a $14,7\times$ u MatMul 1024^2 oproti sekvenční variantě (super-lineární v důsledku anomálně pomalého sekvenčního běhu; i po normalizaci na Parallel(1) zůstává zrychlení super-lineární na vláknech 2–8, přesná příčina nebyla bez profilování určena). Naměřené hodnoty odpovídaly predikci Amdahlova zákona s odhadovaným $P \approx 0,99$ u Game of Life (z 6 fyzických jader). U násobení matic s referencí Parallel(1) se odhad P pohyboval v rozsahu 0,90–0,98. Weak-scaling experiment potvrdil konzistentní slabé škálování u Game of Life (Gustafsonův speedup roste z 1,97 na 2 vláknech na 5,36 na 12 vláknech, efektivita 44,7 %),

zatímco u násobení matic se projeví anomálie sekvenčního běhu (zejména pro velikost 1024^2), které narušují Gustafsonův předpoklad konstantní sekvenční frakce. Konfigurace s 8 a 12 vláken využívají logická vlákna prostřednictvím technologie Hyper-Threading, což negativně ovlivňuje efektivitu paralelizace pro oba algoritmy.

Porovnání knihoven ILGPU a ComputeSharp ukázalo, že ILGPU (CUDA backend) je vhodnější pro iterativní výpočty s velkými daty ($1,7\text{--}6\times$ rychlejší u GoL), zatímco ComputeSharp (DirectX 12) nabízí nižší overhead pro jednorázové kernely a překvapivě dosahuje vyššího výkonu pro největší matice (4096^2), což může souviset s efektivnější správou paměti v DirectX 12 backendu pro velké datové sady. Při implementaci ComputeSharp se ukázalo praktické omezení DirectX 12 v podobě limitu 65 535 thread groups na dimenzi, které vyžadovalo přechod z 1D na 2D dispatch. Po této úpravě ComputeSharp funguje pro všechny testované velikosti. Z hlediska programovacího modelu ComputeSharp využívá source generátory a automatické generování HLSL kódu, což zjednodušuje konfiguraci a spuštění kernelů, ale omezuje flexibilitu oproti ILGPU, které nabízí přímou kontrolu nad alokací, kopírováním a synchronizací. Výhodou ComputeSharp je také platformní nezávislost v rámci Windows – DirectX 12 běží na libovolném zařízení s podporovaným GPU bez nutnosti NVIDIA hardware.

Z práce vyplývá, že C# s knihovnou ILGPU je plně použitelnou alternativou pro GPU výpočty, která nabízí dobrý kompromis mezi pohodlím vývoje a výkonem. Pro produkční nasazení s maximálním důrazem na výkon je ale stále vhodnější nativní CUDA v C/C++, zejména díky možnosti pokročilých optimalizací se sdílenou pamětí.

Možnosti budoucího rozšíření zahrnují testování vlivu manuální konfigurace velikosti bloků vláken na výkon GPU kernelů, implementaci optimalizovaných GPU kernelů se sdílenou pamětí, testování na různém hardware (AMD GPU s OpenCL backendem), přidání dalších typů úloh (Monte Carlo, redukce) a přímé srovnání s nativní CUDA C++ implementací.

Seznam použité literatury

- [1] Amdahl, G. M. (1967). Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities. *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference*, 483–485. <https://doi.org/10.1145/1465482.1465560>
- [2] Anaconda, Inc. (2024). *Numba: A High Performance Python Compiler*. <https://numba.pydata.org/>
- [3] Buck, I., Foley, T., Horn, D., Sugerman, J., Fatahalian, K., Houston, M., & Hanrahan, P. (2004). Brook for GPUs: Stream Computing on Graphics Hardware. *ACM Transactions on Graphics*, 23(3), 777–786. <https://doi.org/10.1145/1015706.1015800>
- [4] Flynn, M. J. (1972). Some Computer Organizations and Their Effectiveness. *IEEE Transactions on Computers*, C-21(9), 948–960. <https://doi.org/10.1109/TC.1972.5009071>
- [5] Gardner, M. (1970). Mathematical Games – The Fantastic Combinations of John Conway’s New Solitaire Game “Life”. *Scientific American*, 223(4), 120–123.
- [6] Gustafson, J. L. (1988). Reevaluating Amdahl’s Law. *Communications of the ACM*, 31(5), 532–533. <https://doi.org/10.1145/42411.42415>
- [7] Hennessy, J. L., & Patterson, D. A. (2017). *Computer Architecture: A Quantitative Approach* (6. vyd.). Morgan Kaufmann.
- [8] Herlihy, M., Shavit, N., Luchangco, V., & Spear, M. (2020). *The Art of Multiprocessor Programming* (2. vyd.). Morgan Kaufmann.
- [9] Hill, M. D., & Marty, M. R. (2008). Amdahl’s Law in the Multicore Era. *Computer*, 41(7), 33–38. <https://doi.org/10.1109/MC.2008.209>
- [10] Khronos Group. (2023). *OpenCL Overview*. <https://www.khronos.org/opencl/>
- [11] Khronos Group. (2024a). *The OpenCL Specification, Version 3.0*. https://registry.khronos.org/OpenCL/specs/3.0-unified/html/OpenCL_API.html
- [12] Khronos Group. (2024b). *Vulkan – Industry Forged*. <https://www.vulkan.org/>
- [13] Kirk, D. B., & Hwu, W.-m. W. (2016). *Programming Massively Parallel Processors: A Hands-on Approach* (3. vyd.). Morgan Kaufmann.
- [14] Klöckner, A. (2024). *PyCUDA*. <https://mathematician.de/software/pycuda/>
- [15] Koester, M. (2024a). *ILGPU – A Modern GPU Compiler for .NET Programs*. <https://github.com/m4rs-mt/ILGPU>
- [16] Koester, M. (2024b). *ILGPU Documentation and Samples*. <https://www.ilgpu.net/>
- [17] Kunz, M. (2024). *ManagedCUDA – A .NET wrapper for the CUDA Driver API*. <https://github.com/kunzmi/managedCuda>

- [18] Lindholm, E., Nickolls, J., Oberman, S., & Montrym, J. (2008). NVIDIA Tesla: A Unified Graphics and Computing Architecture. *IEEE Micro*, 28(2), 39–55. <https://doi.org/10.1109/MM.2008.31>
- [19] Moore, G. E. (1965). Cramming More Components onto Integrated Circuits. *Electronics*, 38(8), 114–117.
- [20] .NET Foundation. (2025). *Silk.NET – High-Speed .NET Multimedia, Graphics, and Compute Library*. <https://github.com/dotnet/Silk.NET>
- [21] Nickolls, J., Buck, I., Garland, M., & Skadron, K. (2008). Scalable Parallel Programming with CUDA. *ACM Queue*, 6(2), 40–53. <https://doi.org/10.1145/1365490.1365500>
- [22] NVIDIA Corporation. (2020). *NVIDIA Ampere GA102 GPU Architecture*. <https://www.nvidia.com/content/PDF/nvidia-ampere-ga-102-gpu-architecture-whitepaper-v2.1.pdf>
- [23] NVIDIA Corporation. (2024a). *CUDA Programming Guide*. <https://docs.nvidia.com/cuda/cuda-programming-guide/>
- [24] NVIDIA Corporation. (2024b). *CUDA Toolkit Documentation*. <https://docs.nvidia.com/cuda/>
- [25] Owens, J. D., Houston, M., Luebke, D., Green, S., Stone, J. E., & Phillips, J. C. (2008). GPU Computing. *Proceedings of the IEEE*, 96(5), 879–899. <https://doi.org/10.1109/JPROC.2008.917757>
- [26] Pedri, S. (2024). *ComputeSharp – A .NET library to run C# code in parallel on the GPU*. <https://github.com/Sergio0694/ComputeSharp>
- [27] Preferred Networks. (2024). *CuPy: NumPy & SciPy for GPU*. <https://cupy.dev/>
- [28] Sanders, J., & Kandrot, E. (2010). *CUDA by Example: An Introduction to General-Purpose GPU Programming*. Addison-Wesley Professional.
- [29] Scarpino, M. (2012). *OpenCL in Action: How to Accelerate Graphics and Computations*. Manning Publications.
- [30] Trobec, R., Slivnik, B., Bulić, P., & Robič, B. (2018). *Introduction to Parallel Computing: From Algorithms to Programming on State-of-the-Art Platforms*. Springer. <https://doi.org/10.1007/978-3-319-98833-7>

Seznam tabulek

2.1	Srovnání architektur CPU a GPU	10
3.1	Srovnání technologií pro GPU výpočty	21
4.1	Hardwarová konfigurace testovacího prostředí	23
4.2	Softwarová konfigurace testovacího prostředí	24
4.3	Škálované velikosti vstupů pro weak-scaling experiment	26
6.1	Časy výpočtu násobení matic [ms] – medián	38
6.2	Časy výpočtu násobení matic – relativní směrodatná odchylka (CV) [%]	39
6.3	Zrychlení CPU paralelní varianty – násobení matic	40
6.4	Zrychlení CPU paralelní varianty normalizované na Parallel(1) – násobení matic	42
6.5	Zrychlení GPU variant oproti sekvenční CPU – násobení matic	43
6.6	Časy výpočtu Game of Life (100 kroků) [ms] – medián	45
6.7	Časy výpočtu Game of Life (100 kroků) – relativní směrodatná odchylka (CV) [%]	45
6.8	Zrychlení CPU paralelní varianty – Game of Life	47
6.9	Zrychlení GPU variant oproti sekvenční CPU – Game of Life	48
6.10	Odhad P z měření na 6 vláknech a porovnání predikce Amdahlova zákona pro 4 jádra (Parallel(1))	50
6.11	Weak-scaling pro násobení matic ($n = 512 \cdot P^{1/3}$)	53
6.12	Weak-scaling pro Game of Life ($n = 512 \cdot \sqrt{P}$)	54
6.13	Porovnání ILGPU a ComputeSharp – časy výpočtu [ms] a poměr	58
B.1	Násobení matic – kompletní naměřené časy (ms), malé matice	99
B.2	Násobení matic – relativní směrodatná odchylka (CV) [%], malé matice	99
B.3	Násobení matic – kompletní naměřené časy pro velké matice (ms)	100
B.4	Násobení matic – relativní směrodatná odchylka (CV) [%] pro velké matice	100
B.5	Násobení matic – speedup paralelní CPU varianty	101
B.6	Game of Life – relativní směrodatná odchylka (CV) [%] pro 100 kroků	101
B.7	Násobení matic – efektivita paralelní CPU varianty ($E = S/N$)	102
B.8	Násobení matic – GPU speedup oproti sekvenční a paralelní variantě	102
B.9	Game of Life – kompletní naměřené časy pro 100 kroků (ms)	103
B.10	Game of Life – speedup paralelní CPU varianty	104

B.11 Game of Life – GPU speedup oproti sekvenční a paralelní variantě	104
B.12 Srovnění naměřeného speedupu s predikcí Amdahlova zákona (strong scaling) . . .	105
B.13 Weak-scaling experiment: Násobení matic – souhrnné statistiky	106
B.14 Weak-scaling experiment: Násobení matic – strict speedup a sekvenční frakce . . .	106
B.15 Weak-scaling experiment: Game of Life – souhrnné statistiky	106
B.16 Weak-scaling experiment: Game of Life – strict speedup a sekvenční frakce	107
B.17 Hardwarová konfigurace testovacího počítače	107
B.18 Softwarová konfigurace	107

Seznam obrázků

6.1	Časy výpočtu násobení matic – porovnání variant (log-log škála)	39
6.2	Zrychlení CPU paralelní varianty vs. počet vláken – násobení matic	41
6.3	Zrychlení CPU paralelní varianty normalizované na Parallel(1) – násobení matic . .	42
6.4	Crossover point – násobení matic (detail pro malé velikosti)	44
6.5	Časy výpočtu Game of Life – porovnání variant (log-log škála)	46
6.6	Zrychlení CPU paralelní varianty vs. počet vláken – Game of Life	47
6.7	GPU zrychlení (ILGPU) obou úloh v závislosti na velikosti vstupu	49
6.8	Porovnání naměřeného zrychlení s predikcí Amdahlova zákona – násobení matic .	51
6.9	Porovnání naměřeného zrychlení s predikcí Amdahlova zákona – Game of Life . .	52
6.10	Weak-scaling čas výpočtu	55
6.11	Zrychlení vůči ideálnímu Gustafsonovu škálování	56
6.12	Odhad sekvenční frakce α podle Gustafsonova zákona	57
6.13	Porovnání ILGPU a ComputeSharp	59

Seznam zkratek

GPU	– Graphics Processing Unit – grafický procesor
CPU	– Central Processing Unit – centrální procesor
GPGPU	– General-Purpose computing on Graphics Processing Units
CUDA	– Compute Unified Device Architecture
OpenCL	– Open Computing Language
SIMD	– Single Instruction, Multiple Data
SIMT	– Single Instruction, Multiple Threads
MIMD	– Multiple Instruction, Multiple Data
ILGPU	– Intermediate Language GPU – knihovna pro GPU výpočty v C#
API	– Application Programming Interface
PCIe	– Peripheral Component Interconnect Express
SM	– Streaming Multiprocessor
VRAM	– Video Random Access Memory
JIT	– Just-In-Time kompilace
GoL	– Game of Life – Hra života

Seznam příloh

Příloha A - Zdrojové kódy

Příloha B - Podrobné výsledky měření

Příloha C - Datové soubory experimentů

Příloha D - Benchmarková aplikace

A Zdrojové kódy

Tato příloha obsahuje kompletní zdrojové kódy implementovaných benchmarků. Úplný projekt včetně konfiguračních souborů a pomocných tříd je dostupný v elektronické příloze práce.

A.1 Rozhraní IBenchmark

Ukázka A.1: Rozhraní pro benchmarkové úlohy

```
public interface IBenchmark
{
    string Name { get; }
    void Setup(int size);
    void Run();
    bool Validate(IBenchmark reference);
    void Cleanup();
    float[] GetResult();
}
```

A.2 Násobení matic – sekvenční verze

Ukázka A.2: Sekvenční násobení matic

```
public class MatrixMultiplySequential : IBenchmark
{
    public string Name => "MatrixMultiply_Sequential";
    private int _size;
    private float[]? _a, _b, _c;

    public void Setup(int size)
    {
        _size = size;
        int total = size * size;
        _a = new float[total];
        _b = new float[total];
        _c = new float[total];
        var rng = new Random(42);
        for (int i = 0; i < total; i++)
        {
            _a[i] = (float)rng.NextDouble();
            _b[i] = (float)rng.NextDouble();
        }
    }

    public void Run()
    {
        int n = _size;
        float[] a = _a!, b = _b!, c = _c!;
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
            {
                float sum = 0f;
                for (int k = 0; k < n; k++)
                    sum += a[i * n + k] * b[k * n + j];
                c[i * n + j] = sum;
            }
    }
}
```

```

public float[] GetResult() => _c!;

public bool Validate(IBenchmark reference)
{
    float[] mine = _c!;
    float[] other = reference.GetResult();
    int checkCount = Math.Min(100, mine.Length);
    for (int i = 0; i < checkCount; i++)
        if (Math.Abs(mine[i] - other[i]) > 1e-2f)
            return false;
    return true;
}

public void Cleanup()
{
    _a = null; _b = null; _c = null;
}
}

```

A.3 Násobení matic – paralelní CPU verze

Ukázka A.3: Paralelní CPU násobení matic

```

public class MatrixMultiplyParallel : IBenchmark
{
    private readonly int _threadCount;
    private int _size;
    private float[]? _a, _b, _c;

    public string Name =>
        $"MatrixMultiplyParallel_{_threadCount}";

    public MatrixMultiplyParallel(int threadCount)
    {
        _threadCount = threadCount;
    }
}

```

```

}

public void Setup(int size)
{
    _size = size;
    int total = size * size;
    _a = new float[total];
    _b = new float[total];
    _c = new float[total];
    var rng = new Random(42);
    for (int i = 0; i < total; i++)
    {
        _a[i] = (float)rng.NextDouble();
        _b[i] = (float)rng.NextDouble();
    }
}

public void Run()
{
    int n = _size;
    float[] a = _a!, b = _b!, c = _c!;
    var options = new ParallelOptions
        { MaxDegreeOfParallelism = _threadCount };

    Parallel.For(0, n, options, i =>
    {
        for (int j = 0; j < n; j++)
        {
            float sum = 0f;
            for (int k = 0; k < n; k++)
                sum += a[i * n + k] * b[k * n + j];
            c[i * n + j] = sum;
        }
    });
}

```

```

public float[] GetResult() => _c!;

public bool Validate(IBenchmark reference)
{
    float[] mine = _c!;
    float[] other = reference.GetResult();
    int checkCount = Math.Min(100, mine.Length);
    for (int i = 0; i < checkCount; i++)
        if (Math.Abs(mine[i] - other[i]) > 1e-2f)
            return false;
    return true;
}

public void Cleanup()
{
    _a = null; _b = null; _c = null;
}
}

```

A.4 Násobení matic – GPU verze

Ukázka A.4: GPU násobení matic pomocí ILGPU

```

public class MatrixMultiplyGpu : IBenchmark
{
    public string Name => "MatrixMultiply_GPU";
    private int _size;
    private float[]? _a, _b, _c;
    private Context? _context;
    private Accelerator? _accelerator;
    private Action<Index1D,
        ArrayView1D<float, Stride1D.Dense>,
        ArrayView1D<float, Stride1D.Dense>,
        ArrayView1D<float, Stride1D.Dense>,
        int>? _kernel;

```

```

public void Setup(int size)
{
    _size = size;
    int total = size * size;
    _a = new float[total];
    _b = new float[total];
    _c = new float[total];
    var rng = new Random(42);
    for (int i = 0; i < total; i++)
    {
        _a[i] = (float)rng.NextDouble();
        _b[i] = (float)rng.NextDouble();
    }

    (_context, _accelerator) =
        GpuSelector.CreateAccelerator();
    _kernel = _accelerator
        .LoadAutoGroupedStreamKernel<
            Index1D,
            ArrayView1D<float, Stride1D.Dense>,
            ArrayView1D<float, Stride1D.Dense>,
            ArrayView1D<float, Stride1D.Dense>,
            int>(MatMulKernel);
}

static void MatMulKernel(
    Index1D index,
    ArrayView1D<float, Stride1D.Dense> a,
    ArrayView1D<float, Stride1D.Dense> b,
    ArrayView1D<float, Stride1D.Dense> c,
    int n)
{
    int row = index / n;
    int col = index % n;

```

```

        float sum = 0f;
        for (int k = 0; k < n; k++)
            sum += a[row * n + k] * b[k * n + col];
        c[row * n + col] = sum;
    }

    public void Run()
    {
        int n = _size;
        var acc = _accelerator!;
        using var bufA = acc.Allocate1D<float>(_a!.Length);
        using var bufB = acc.Allocate1D<float>(_b!.Length);
        using var bufC = acc.Allocate1D<float>(_c!.Length);

        bufA.CopyFromCPU(_a!);
        bufB.CopyFromCPU(_b!);

        _kernel!(n * n, bufA.View, bufB.View,
            bufC.View, n);
        acc.Synchronize();
        bufC.CopyToCPU(_c!);
    }

    public float[] GetResult() => _c!;

    public bool Validate(IBenchmark reference)
    {
        float[] mine = _c!;
        float[] other = reference.GetResult();
        int checkCount = Math.Min(100, mine.Length);
        for (int i = 0; i < checkCount; i++)
            if (Math.Abs(mine[i] - other[i]) > 1e-2f)
                return false;
        return true;
    }

```

```
public void Cleanup()
{
    _kernel = null;
    _accelerator?.Dispose();
    _context?.Dispose();
    _a = null; _b = null; _c = null;
}
}
```

A.5 Game of Life – sekvenční verze

Ukázka A.5: Sekvenční Game of Life

```
public class GameOfLifeSequential : IBenchmark
{
    public string Name => "GameOfLife_Sequential";
    private int _size;
    private int[]? _current, _next;

    public void Setup(int size)
    {
        _size = size;
        int total = size * size;
        _current = new int[total];
        _next = new int[total];
        var rng = new Random(42);
        for (int i = 0; i < total; i++)
            _current[i] = rng.Next(2);
    }

    public void Run()
    {
        int n = _size;
        int[] current = _current!;
        int[] next = _next!;
```

```

for (int step = 0; step < 100; step++)
{
    for (int y = 0; y < n; y++)
    for (int x = 0; x < n; x++)
    {
        int neighbors = 0;
        for (int dy = -1; dy <= 1; dy++)
        for (int dx = -1; dx <= 1; dx++)
        {
            if (dx == 0 && dy == 0) continue;
            int nx = (x + dx + n) % n;
            int ny = (y + dy + n) % n;
            neighbors += current[ny * n + nx];
        }
        bool alive = current[y * n + x] == 1;
        next[y * n + x] = alive
            ? ((neighbors == 2 || neighbors == 3)
               ? 1 : 0)
            : (neighbors == 3 ? 1 : 0);
    }
    var tmp = current;
    current = next;
    next = tmp;
}
_current = current;
_next = next;
}

```

```

public float[] GetResult()
{
    var result = new float[_current!.Length];
    for (int i = 0; i < _current.Length; i++)
        result[i] = _current[i];
    return result;
}

```

```

    }

    public bool Validate(IBenchmark reference)
    {
        float[] mine = GetResult();
        float[] other = reference.GetResult();
        if (mine.Length != other.Length) return false;
        for (int i = 0; i < mine.Length; i++)
            if ((int)mine[i] != (int)other[i])
                return false;
        return true;
    }

    public void Cleanup()
    {
        _current = null; _next = null;
    }
}

```

A.6 Game of Life – paralelní CPU verze

Ukázka A.6: Paralelní CPU Game of Life

```

public class GameOfLifeParallel : IBenchmark
{
    private readonly int _threadCount;
    private int _size;
    private int[]? _current, _next;

    public string Name =>
        $"GameOfLife_Parallel_{_threadCount}";

    public GameOfLifeParallel(int threadCount)
    {
        _threadCount = threadCount;
    }
}

```

```

public void Setup(int size)
{
    _size = size;
    int total = size * size;
    _current = new int[total];
    _next = new int[total];
    var rng = new Random(42);
    for (int i = 0; i < total; i++)
        _current[i] = rng.Next(2);
}

public void Run()
{
    int n = _size;
    int[] current = _current!;
    int[] next = _next!;
    var options = new ParallelOptions
        { MaxDegreeOfParallelism = _threadCount };

    for (int step = 0; step < 100; step++)
    {
        int[] src = current, dst = next;
        Parallel.For(0, n, options, y =>
        {
            for (int x = 0; x < n; x++)
            {
                int neighbors = 0;
                for (int dy = -1; dy <= 1; dy++)
                    for (int dx = -1; dx <= 1; dx++)
                    {
                        if (dx == 0 && dy == 0) continue;
                        int nx = (x + dx + n) % n;
                        int ny = (y + dy + n) % n;
                        neighbors += src[ny * n + nx];
                    }
            }
        });
        current = next;
        next = src;
    }
}

```

```

    }
    bool alive = src[y * n + x] == 1;
    dst[y * n + x] = alive
        ? ((neighbors == 2 || neighbors == 3)
           ? 1 : 0)
        : (neighbors == 3 ? 1 : 0);
    }
});
var tmp = current;
current = next;
next = tmp;
}
_current = current;
_next = next;
}

public float[] GetResult()
{
    var result = new float[_current!.Length];
    for (int i = 0; i < _current.Length; i++)
        result[i] = _current[i];
    return result;
}

public bool Validate(IBenchmark reference)
{
    float[] mine = GetResult();
    float[] other = reference.GetResult();
    if (mine.Length != other.Length) return false;
    for (int i = 0; i < mine.Length; i++)
        if ((int)mine[i] != (int)other[i])
            return false;
    return true;
}

```

```

public void Cleanup()
{
    _current = null; _next = null;
}
}

```

A.7 Game of Life – GPU verze

Ukázka A.7: GPU Game of Life pomocí ILGPU

```

public class GameOfLifeGpu : IBenchmark
{
    public string Name => "GameOfLife_GPU";
    private int _size;
    private int[]? _initialGrid, _result;
    private Context? _context;
    private Accelerator? _accelerator;
    private Action<Index1D,
        ArrayView1D<int, Stride1D.Dense>,
        ArrayView1D<int, Stride1D.Dense>,
        int>? _kernel;

    public void Setup(int size)
    {
        _size = size;
        int total = size * size;
        _initialGrid = new int[total];
        _result = new int[total];
        var rng = new Random(42);
        for (int i = 0; i < total; i++)
            _initialGrid[i] = rng.Next(2);

        (_context, _accelerator) =
            GpuSelector.CreateAccelerator();
        _kernel = _accelerator
            .LoadAutoGroupedStreamKernel<

```

```

        Index1D,
        ArrayView1D<int, Stride1D.Dense>,
        ArrayView1D<int, Stride1D.Dense>,
        int>(GoLKernel);
}

```

```

static void GoLKernel(
    Index1D index,
    ArrayView1D<int, Stride1D.Dense> current,
    ArrayView1D<int, Stride1D.Dense> next,
    int n)
{
    int y = index / n;
    int x = index % n;
    int neighbors = 0;
    for (int dy = -1; dy <= 1; dy++)
    for (int dx = -1; dx <= 1; dx++)
    {
        if (dx == 0 && dy == 0) continue;
        int nx = (x + dx + n) % n;
        int ny = (y + dy + n) % n;
        neighbors += current[ny * n + nx];
    }
    bool alive = current[y * n + x] == 1;
    next[y * n + x] = alive
        ? ((neighbors == 2 || neighbors == 3)
           ? 1 : 0)
        : (neighbors == 3 ? 1 : 0);
}

```

```

public void Run()
{
    int n = _size;
    var acc = _accelerator!;
    int total = n * n;

```

```

using var bufA = acc.Allocate1D<int>(total);
using var bufB = acc.Allocate1D<int>(total);
bufA.CopyFromCPU(_initialGrid!);

var current = bufA;
var next = bufB;
for (int step = 0; step < 100; step++)
{
    _kernel!(total, current.View,
              next.View, n);
    acc.Synchronize();
    var tmp = current;
    current = next;
    next = tmp;
}
current.CopyToCPU(_result!);
}

public float[] GetResult()
{
    var result = new float[_result!.Length];
    for (int i = 0; i < _result.Length; i++)
        result[i] = _result[i];
    return result;
}

public bool Validate(IBenchmark reference)
{
    float[] mine = GetResult();
    float[] other = reference.GetResult();
    if (mine.Length != other.Length) return false;
    for (int i = 0; i < mine.Length; i++)
        if ((int)mine[i] != (int)other[i])
            return false;
}

```

```

        return true;
    }

    public void Cleanup()
    {
        _kernel = null;
        _accelerator?.Dispose();
        _context?.Dispose();
        _initialGrid = null; _result = null;
    }
}

```

A.8 Třída BenchmarkRunner

Ukázka A.8: Hlavní spouštěč benchmarků

```

public class BenchmarkRunner
{
    private const int WarmupRuns = 3;
    private const int DefaultMeasuredRuns = 10;

    public int WarmupOverride { get; init; } = WarmupRuns;
    public int MeasuredRunsOverride { get; init; }
        = DefaultMeasuredRuns;

    public List<BenchmarkResult> Run(
        IBenchmark benchmark,
        IBenchmark? referenceForValidation,
        string taskName, string variant, int size)
    {
        // Zahřívací behy
        for (int i = 0; i < WarmupOverride; i++)
        {
            benchmark.Setup(size);
            benchmark.Run();
            benchmark.Cleanup();
        }
    }
}

```

```

}

// Sonda pro odhad doby jednoho behu
benchmark.Setup(size);
var probe = Stopwatch.StartNew();
benchmark.Run();
probe.Stop();
benchmark.Cleanup();

int measuredRuns = MeasuredRunsOverride;
double estimatedMs = probe.Elapsed.TotalMilliseconds;
if (MeasuredRunsOverride == DefaultMeasuredRuns)
{
    if (estimatedMs > 60_000) measuredRuns = 3;
    else if (estimatedMs > 30_000) measuredRuns = 5;
}

// Reference pro validaci
if (referenceForValidation != null)
{
    referenceForValidation.Setup(size);
    referenceForValidation.Run();
}

var results = new List<BenchmarkResult>();
var sw = new Stopwatch();
bool? validationResult = null;

for (int i = 0; i < measuredRuns; i++)
{
    benchmark.Setup(size);
    sw.Restart();
    benchmark.Run();
    sw.Stop();

```

```

        if (i == 0 && referenceForValidation != null)
            validationResult =
                benchmark.Validate(referenceForValidation);

        results.Add(new BenchmarkResult
        {
            TaskName = taskName,
            Variant = variant,
            InputSize = size,
            RunNumber = i + 1,
            TimeMs = sw.Elapsed.TotalMilliseconds,
            IsValid = validationResult ?? true
        });
        benchmark.Cleanup();
    }

    referenceForValidation?.Cleanup();
    return results;
}

public class BenchmarkResult
{
    public string TaskName { get; set; }
    public string Variant { get; set; }
    public int InputSize { get; set; }
    public int RunNumber { get; set; }
    public double TimeMs { get; set; }
    public bool IsValid { get; set; }
}

```

A.9 Násobení matic – ComputeSharp verze

Ukázka A.9: ComputeSharp násobení matic (DirectX 12)

```
public class MatrixMultiplyComputeSharp : IBenchmark
{
    public string Name => "MatrixMultiply_GPU_ComputeSharp";
    private int _size;
    private float[]? _a, _b, _c;

    public void Setup(int size)
    {
        _size = size;
        int total = size * size;
        _a = new float[total];
        _b = new float[total];
        _c = new float[total];
        var rng = new Random(42);
        for (int i = 0; i < total; i++)
        {
            _a[i] = (float)rng.NextDouble();
            _b[i] = (float)rng.NextDouble();
        }
    }

    public void Run()
    {
        int n = _size;
        var device = GraphicsDevice.Default();
        using var bufA =
            device.AllocateReadOnlyBuffer<float>(_a!);
        using var bufB =
            device.AllocateReadOnlyBuffer<float>(_b!);
        using var bufC =
            device.AllocateReadWriteBuffer<float>(n * n);
```

```

        device.For(n, n,
            new MatMulShader(bufA, bufB, bufC, n));
        bufC.CopyTo(_c!);
    }

    // Validate, GetResult, Cleanup -- shodne s ILGPU verzi
    public float[] GetResult() => _c!;
    public bool Validate(IBenchmark reference) { ... }
    public void Cleanup() { _a = _b = _c = null; }
}

[ThreadGroupSize(DefaultThreadGroupSizes.X)]
[GeneratedComputeShaderDescriptor]
internal readonly partial struct MatMulShader
    : IComputeShader
{
    public readonly ReadOnlyBuffer<float> A;
    public readonly ReadOnlyBuffer<float> B;
    public readonly ReadWriteBuffer<float> C;
    public readonly int N;

    public void Execute()
    {
        int row = ThreadIds.Y;
        int col = ThreadIds.X;
        float sum = 0f;
        for (int k = 0; k < N; k++)
            sum += A[row * N + k] * B[k * N + col];
        C[row * N + col] = sum;
    }
}

```

A.10 Game of Life – ComputeSharp verze

Ukázka A.10: ComputeSharp Game of Life (DirectX 12) s 2D dispatchem

```
public class GameOfLifeComputeSharp : IBenchmark
{
    public string Name => "GameOfLife_GPU_ComputeSharp";
    private int _size;
    private int[]? _initialGrid;
    private float[]? _result;

    public void Setup(int size)
    {
        _size = size;
        int total = size * size;
        _initialGrid = new int[total];
        var rng = new Random(42);
        for (int i = 0; i < total; i++)
            _initialGrid[i] = rng.Next(2);
    }

    public void Run()
    {
        int n = _size;
        var device = GraphicsDevice.GetDefault();
        using var bufA =
            device.AllocateReadWriteBuffer<int>(
                _initialGrid!);
        using var bufB =
            device.AllocateReadWriteBuffer<int>(n * n);

        // 2D dispatch: device.For(n, n, shader)
        for (int step = 0; step < 100; step++)
        {
            if (step % 2 == 0)
                device.For(n, n,
```

```

        new GoLShader(bufA, bufB, n));
    else
        device.For(n, n,
            new GoLShader(bufB, bufA, n));
}

int[] raw = new int[n * n];
bufA.CopyTo(raw);
_result = new float[n * n];
for (int i = 0; i < n * n; i++)
    _result[i] = raw[i];
}

// Validate, GetResult, Cleanup -- shodne s ILGPU verzi
public float[] GetResult() => _result!;
public bool Validate(IBenchmark reference) { ... }
public void Cleanup()
{ _initialGrid = null; _result = null; }
}

[ThreadGroupSize(DefaultThreadGroupSizes.X)]
[GeneratedComputeShaderDescriptor]
internal readonly partial struct GoLShader
    : IComputeShader
{
    public readonly ReadWriteBuffer<int> Current;
    public readonly ReadWriteBuffer<int> Next;
    public readonly int N;

    public void Execute()
    {
        int x = ThreadIds.Y;
        int y = ThreadIds.X;
        int neighbors = 0;
        for (int dx = -1; dx <= 1; dx++)

```

```
for (int dy = -1; dy <= 1; dy++)
{
    if (dx == 0 && dy == 0) continue;
    int nx = (x + dx + N) % N;
    int ny = (y + dy + N) % N;
    neighbors += Current[nx * N + ny];
}
bool alive = Current[x * N + y] == 1;
Next[x * N + y] = alive
    ? ((neighbors == 2 || neighbors == 3)
       ? 1 : 0)
    : (neighbors == 3 ? 1 : 0);
}
}
```

B Podrobné výsledky měření

Tato příloha obsahuje kompletní tabulky naměřených hodnot pro všechny testované konfigurace včetně relativní směrodatné odchylky (koeficientu variace, *CV*) v procentech.

B.1 Násobení matic – kompletní časy

Tabulka č. B.1: Násobení matic – kompletní naměřené časy (ms), malé matice

Varianta	64	128	256	512	1024
Sekvenční CPU	0,19	2,31	17,98	186,68	5274,57
Paralelní CPU (1)	0,58	2,84	18,30	201,00	4054,45
Paralelní CPU (2)	0,30	1,46	9,23	107,28	1819,08
Paralelní CPU (4)	0,19	0,75	4,73	64,24	909,47
Paralelní CPU (6)	0,19	0,54	4,13	49,18	606,06
Paralelní CPU (8)	0,19	0,65	4,11	44,95	489,24
Paralelní CPU (max)	0,19	0,61	4,15	36,90	359,35
GPU ILGPU	3,04	3,18	3,28	4,11	8,18
GPU ComputeSharp	2,20	2,39	3,05	3,60	29,32

Zdroj: vlastní zpracování

Tabulka č. B.2: Násobení matic – relativní směrodatná odchylka (*CV*) [%], malé matice

Varianta	64	128	256	512	1024
Sekvenční CPU	1,01	0,79	2,85	0,71	3,59
Paralelní CPU (1)	2,51	0,78	1,07	1,25	2,82
Paralelní CPU (2)	5,75	6,23	1,49	5,39	3,45
Paralelní CPU (4)	5,89	4,77	14,74	6,84	2,85
Paralelní CPU (6)	7,54	17,08	7,84	3,57	3,10
Paralelní CPU (8)	6,62	3,66	5,28	2,34	3,36
Paralelní CPU (max)	9,26	10,51	8,31	3,21	3,62
GPU ILGPU	2,56	3,64	2,08	4,88	5,61
GPU ComputeSharp	3,66	6,11	6,52	98,80	3,65

Zdroj: vlastní zpracování

Tabulka č. B.3: Násobení matic – kompletní naměřené časy pro velké matice (ms)

Varianta	1024	2048	4096
Sekvenční CPU	5 274	69 106	1 463 460
Paralelní CPU (1)	4 054	45 256	639 405
Paralelní CPU (2)	1 819	22 502	400 063
Paralelní CPU (4)	909	11 656	245 441
Paralelní CPU (6)	606	8 284	142 600
Paralelní CPU (8)	489	7 963	184 263
Paralelní CPU (max)	359	7 675	144 041
GPU ILGPU	8,18	34,59	640,33
GPU ComputeSharp	29,32	49,49	223,96

Zdroj: vlastní zpracování

Tabulka č. B.4: Násobení matic – relativní směrodatná odchylka (CV) [%] pro velké matice

Varianta	1024	2048	4096
Sekvenční CPU	3,59	21,93	8,38
Paralelní CPU (1)	2,82	31,66	15,51
Paralelní CPU (2)	3,45	6,41	11,43
Paralelní CPU (4)	2,85	45,71	20,77
Paralelní CPU (6)	3,10	52,88	17,67
Paralelní CPU (8)	3,36	49,49	4,29
Paralelní CPU (max)	3,62	50,74	15,34
GPU ILGPU	5,61	12,75	0,83
GPU ComputeSharp	3,65	7,36	16,34

Zdroj: vlastní zpracování

B.2 Násobení matic – speedup a efektivita

Tabulka č. B.5: Násobení matic – speedup paralelní CPU varianty

Vlákna	64	128	256	512	1024	2048	4096
1	0,33	0,81	0,94	0,93	1,30	1,53	2,29
2	0,64	1,58	1,87	1,74	2,90	3,07	3,66
4	1,00	3,07	3,65	2,91	5,80	5,93	5,96
6	1,01	4,31	4,18	3,80	8,70	8,34	10,26
8	1,01	3,58	4,20	4,16	10,78	8,68	7,94
12	1,02	3,80	4,16	5,06	14,68	9,00	10,16

Zdroj: vlastní zpracování

Tabulka č. B.6: Game of Life – relativní směrodatná odchylka (CV) [%] pro 100 kroků

Varianta	8	32	100	500	1000	2000	5000	10 000
Sekvenční CPU	0,26	0,11	0,03	0,02	0,03	0,04	0,01	0,01
Paralelní CPU (1)	2,11	0,24	0,11	0,04	0,03	0,03	0,02	0,03
Paralelní CPU (2)	8,46	0,58	0,09	0,32	0,14	0,11	0,05	0,02
Paralelní CPU (4)	147,23	22,50	0,61	0,26	0,19	0,21	0,06	0,04
Paralelní CPU (6)	6,26	2,95	1,72	5,80	0,46	0,55	0,23	0,08
Paralelní CPU (8)	10,03	37,32	2,90	0,47	0,78	0,32	0,15	0,13
Paralelní CPU (max)	3,05	1,93	2,58	0,18	0,06	0,05	0,04	0,02
GPU ILGPU	4,23	2,48	21,24	15,15	17,86	7,80	2,44	1,10
GPU ComputeSharp	1,15	3,20	2,05	2,49	31,19	96,95	2,46	7,00

Zdroj: vlastní zpracování

Tabulka č. B.7: Násobení matic – efektivita paralelní CPU varianty ($E = S/N$)

Vlákna	64	128	256	512	1024	2048	4096
2	0,32	0,79	0,94	0,87	1,45	1,54	1,83
4	0,25	0,77	0,91	0,73	1,45	1,48	1,49
6	0,17	0,72	0,70	0,63	1,45	1,39	1,71
8	0,13	0,45	0,52	0,52	1,35	1,08	0,99
12	0,08	0,32	0,35	0,42	1,22	0,75	0,85

Zdroj: vlastní zpracování

Tabulka č. B.8: Násobení matic – GPU speedup oproti sekvenční a paralelní variantě

Velikost	GPU čas (ms)	Speedup vs. seq.	Speedup vs. par. (max)
64	3,04	0,06	0,06
128	3,20	0,72	0,19
256	3,05	5,89	1,35
512	3,60	51,86	17,85
1024	8,18	644,69	43,93
2048	34,59	2005,04	222,70
4096	654,41	6534,25	571,86

Zdroj: vlastní zpracování

B.3 Game of Life – kompletní časy

Tabulka č. B.9: Game of Life – kompletní naměřené časy pro 100 kroků (ms)

Varianta	8	32	100	500	1000	2000	5000	10 000
Sekvenční CPU	0,38	6,16	57,42	1 437	5 750	23 020	143 819	575 307
Paralelní CPU (1)	0,74	5,94	57,57	1 439	5 758	23 040	143 936	576 011
Paralelní CPU (2)	0,53	3,38	29,14	722	2 884	11 544	72 048	288 134
Paralelní CPU (4)	0,79	1,92	15,15	362	1 454	5 797	36 225	144 487
Paralelní CPU (6)	0,68	1,83	13,32	249	1 004	3 994	24 773	98 595
Paralelní CPU (8)	0,72	1,87	13,95	272	1 022	4 016	24 562	97 676
Paralelní CPU (max)	0,70	1,89	13,11	258	971	3 871	23 919	95 333
GPU ILGPU	4,40	4,42	4,34	5,26	8,36	22,55	98,47	368,47
GPU ComputeSharp	9,00	8,87	8,40	17,02	50,54	38,27	166,63	788,81

Zdroj: vlastní zpracování

B.4 Game of Life – speedup a efektivita

Tabulka č. B.10: Game of Life – speedup paralelní CPU varianty

Vlákna	8	32	100	500	1000	2000	5000	10 000
1	0,51	0,99	1,00	1,00	1,00	1,00	1,00	1,00
2	0,71	1,74	1,97	1,99	1,99	1,99	2,00	2,00
4	0,48	3,07	3,79	3,97	3,96	3,97	3,97	3,98
6	0,55	3,22	4,31	5,78	5,73	5,76	5,81	5,84
8	0,53	3,14	4,12	5,28	5,63	5,73	5,86	5,89
12	0,54	3,10	4,38	5,58	5,92	5,95	6,01	6,03

Zdroj: vlastní zpracování

Tabulka č. B.11: Game of Life – GPU speedup oproti sekvenční a paralelní variantě

Mřížka	GPU čas (ms)	Speedup vs. seq.	Speedup vs. par. (max)
8	4,40	0,09	0,16
32	4,42	1,33	0,43
100	4,34	13,24	3,02
500	5,26	273,25	48,96
1000	8,36	688,13	116,21
2000	22,55	1020,70	171,63
5000	98,47	1460,57	243,00
10 000	368,47	1561,35	258,75

Zdroj: vlastní zpracování

B.5 Amdahlova analýza – strong-scaling experiment

Tabulka č. B.12: Srovnění naměřeného speedupu s predikcí Amdahlova zákona (strong scaling)

Úloha	P (odhad)	Teor. $S(8)$	Skut. $S(8)$	Odchylka (%)
Matice 256^2	0,91	4,96	4,20	15,5
Matice 512^2	0,88	4,42	4,16	5,9
Matice 1024^2	1,00	8,00	10,78	34,8
Matice 2048^2	1,00	8,00	8,68	8,5
Matice 4096^2	1,00	8,00	7,94	0,7
GoL 500^2	0,99	7,60	5,28	30,6
GoL 1000^2	0,99	7,50	5,63	25,0
GoL 2000^2	0,99	7,57	5,73	24,2
GoL 5000^2	0,99	7,64	5,86	23,4
GoL 10000^2	0,99	7,70	5,89	23,5

Zdroj: vlastní zpracování

Tato tabulka obsahuje výsledky silného škálování (fixní velikost vstupu), ve kterém se počet procesorů zvyšuje. Naměřené speedupy jsou porovnány s předpovědí Amdahlova zákona pro odhadnutou sekvenční frakci P .

B.6 Gustafsonova analýza – weak-scaling experiment

Následující tabulky obsahují kompletní výsledky weak-scaling experimentu, ve kterém se velikost vstupu škáluje s počtem procesorů podle vzorců $n = 512 \cdot P^{1/3}$ (násobení matic) a $n = 512 \cdot \sqrt{P}$ (Game of Life). Pro každou konfiguraci bylo provedeno 10 běhů.

B.6.1 Násobení matic – weak-scaling výsledky

Tabulka č. B.13: Weak-scaling experiment: Násobení matic – souhrnné statistiky

Vlákná	Velikost	Median [ms]	Průměr [ms]	SD [ms]	CV [%]	Běhy
2	645 ²	145,7	146,9	4,84	3,29	10
4	813 ²	177,3	177,6	3,75	2,11	10
6	930 ²	229,6	230,4	18,86	8,18	10
8	1024 ²	463,6	470,6	14,78	3,14	10
12	1172 ²	593,3	592,8	27,20	4,59	10

Zdroj: vlastní zpracování

Tabulka č. B.14: Weak-scaling experiment: Násobení matic – strict speedup a sekvenční frakce

Vlákná	Velikost	Seq. čas [ms]	$S_G(N)$	α_{strict}	Efektivita [%]
2	645 ²	230,7	1,58	0,42	79,0
4	813 ²	522,9	2,95	0,35	73,8
6	930 ²	741,1	3,23	0,55	53,8
8	1024 ²	5038,9	10,87	-0,41	135,9
12	1172 ²	1543,1	2,60	0,85	21,7

Zdroj: vlastní zpracování

B.6.2 Game of Life – weak-scaling výsledky

Tabulka č. B.15: Weak-scaling experiment: Game of Life – souhrnné statistiky

Vlákná	Velikost	Median [ms]	Průměr [ms]	SD [ms]	CV [%]	Běhy
2	724 ²	1566,7	1566,6	20,17	1,29	10
4	1024 ²	1775,2	1784,6	53,40	2,99	10
6	1254 ²	2068,6	2072,0	34,13	1,65	10
8	1448 ²	2321,5	2327,9	28,97	1,24	10
12	1774 ²	3386,7	3437,2	182,82	5,32	10

Zdroj: vlastní zpracování

Tabulka č. B.16: Weak-scaling experiment: Game of Life – strict speedup a sekvenční frakce

Vlákna	Velikost	Seq. čas [ms]	$S_G(N)$	α_{strict}	Efektivita [%]
2	724 ²	3088,5	1,97	0,029	98,5
4	1024 ²	6153,3	3,47	0,18	86,8
6	1254 ²	9234,7	4,46	0,31	74,3
8	1448 ²	12 130,1	5,23	0,40	65,4
12	1774 ²	18 162,7	5,36	0,603	44,7

Zdroj: vlastní zpracování

B.7 Konfigurace testovacího prostředí

Tabulka č. B.17: Hardwarová konfigurace testovacího počítače

Komponenta	Specifikace
Procesor	AMD Ryzen 5 3600 (6 jader / 12 vláken, 3,6 GHz)
RAM	32 GB DDR4
GPU	NVIDIA GeForce RTX 3070 (GA104)
GPU paměť	8 GB GDDR6
CUDA jádra	5888
CUDA verze	Compute Capability 8.6 (Ampere)
Disk	NVMe SSD

Zdroj: vlastní zpracování

Tabulka č. B.18: Softwarová konfigurace

Software	Verze
OS	Windows 10 (build 19045)
.NET	10.0.5 (C# 14)
ILGPU	1.5.3 (CUDA PTX backend)
ComputeSharp	3.1.0 (DirectX 12 backend)
GPU driver	NVIDIA Game Ready

Zdroj: vlastní zpracování

C Datové soubory experimentů

Elektronická příloha `results.zip` obsahuje naměřená data z experimentů ve formátu CSV (UTF-8, oddělovač čárka, desetinná tečka). Soubory zahrnují surová data i odvozené analýzy pro silné i slabé škálování – konkrétně soubory `raw_results.csv` a `summary.csv` se surovými a souhrnnými daty silného škálování, `parallel1_analysis.csv` a `amdahl_analysis.csv` s analýzou zrychlení a Amdahlova zákona, a odpovídající soubory pro slabé škálování s prefixem `weak_` včetně analýzy dle Gustafsonova zákona.

D Benchmarková aplikace

Elektronická příloha `benchmarkingApp.zip` obsahuje kompletní .NET solution (`GpuBenchmarks.sln`) benchmarkové aplikace v jazyce C#. Projekt implementuje sekvenční, paralelní (CPU) a GPU varianty (ILGPU, ComputeSharp) úloh násobení matic a Game of Life popsaných v kapitola 5. Aplikace podporuje silné i slabé škálování a automaticky generuje CSV výstupy s naměřenými daty a odvozenými analýzami.

Požadavky na spuštění a pokyny ke kompilaci jsou popsány v souboru `README.md`. Ke kompilaci je potřeba .NET 10.0 SDK a GPU s podporou CUDA/Vulkan (ILGPU) nebo DirectX 12 (ComputeSharp).